Université d'Évry – Paris-Saclay
Master 1 – Computer and Network Systems

Travail d'Étude et de Recherche – Année 2025/2026

Minimisation de l'énergie par ajustement de vitesse assistée par l'apprentissage

Étudiants :
Taha Abdelkrim Douik
20254376@etud.univ-evry.fr

Mohamed Amine Menadi
20254852@etud.univ-evry.fr

Encadrant :
Vincent Chau
vincent.chau@univ-evry.fr

20 mai 2026

Résumé

Résumé. L'optimisation énergétique des processeurs modernes repose sur la technique du *Dynamic Voltage Scaling* (DVS), qui ajuste dynamiquement la vitesse d'exécution pour minimiser la consommation $\int s(t)^\alpha \, dt$. Dans le modèle d'ordonnancement de Yao, Demers et Shenker, un ensemble de tâches caractérisées par un triplet (r_i, d_i, w_i) doit être exécuté sans dépasser les échéances. Si l'algorithme optimal hors-ligne YDS est connu depuis 1995, l'ordonnancement *en ligne* reste un défi : les meilleurs algorithmes classiques (AVR, OA, BKP) ne dépassent pas un certain ratio compétitif. Les travaux récents de Bamas et al. (2020) introduisent un algorithme d'ordonnancement augmenté par une prédiction (LAS), paramétré par un facteur de confiance ϵ ; cet algorithme atteint un ratio quasi-optimal lorsque la prédiction est fiable, mais s'effondre (ratio observé jusqu'à 1,447, supérieur à OA à 1,118 sur la même instance) lorsqu'elle est erronée.

Ce rapport propose un **Moteur de Confiance Dynamique** qui mesure en temps réel l'erreur de la prédiction et ajuste dynamiquement, à travers un niveau de confiance c, la part de la prédiction transmise à LAS. Nos expériences, conduites sur des instances synthétiques et sur le jeu de données réel *Brightkite* [CML11], montrent que ce mécanisme préserve le ratio optimal ($\approx 1{,}00$) quand la prédiction est de bonne qualité, plafonne la dégradation à 1,100 dans les pires scénarios adversariaux (contre 1,447 pour LAS classique), et apporte un **gain énergétique de** 4,13% sur trafic réel mesuré sur 200 jours consécutifs (ratio moyen passant de 1,1261 à 1,0796, $p < 10^{-6}$ par test de Wilcoxon apparié). Le moteur réduit en outre l'écart-type jour à jour d'un facteur 4, ce qui traduit une stabilité bien supérieure de la consommation énergétique.

Table des matières

Résumé **1**

1 Introduction **4**
1.1 Contexte : énergie et calcul 4
1.2 Le problème d'ordonnancement éco-énergétique 4
Objectif du Travail d'étude et de recherche 5
1.3 De l'algorithme en ligne aux prédictions apprises 5
1.4 Problématique et contribution 6
1.5 Plan du rapport 6

2 Modèle et préliminaires **8**
2.1 Modèle d'ordonnancement avec speed scaling 8
2.2 Algorithmes en ligne et ratio compétitif 9
2.3 Cadre des Learning-Augmented Algorithms 10

3 État de l'art **12**
3.1 YDS : l'algorithme optimal hors-ligne 12
3.2 AVR (Average Rate) 13
3.3 OA (Optimal Available) 13
3.4 BKP (Bansal–Kimbrel–Pruhs) 13
3.5 LAS : Learning-Augmented Scheduling 14
3.5.1 Données d'entrée et paramètre ϵ 14
3.5.2 Principe en quatre étapes 14
3.5.3 Garanties théoriques 15

4 Limites de LAS et motivation **16**
4.1 Le compromis statique de ϵ 16
4.2 Validation expérimentale du défaut 16
4.3 Origine structurelle du problème 17
4.4 Question de recherche 17

5 Un Moteur de Confiance Dynamique pour LAS **19**
5.1 Principe général 19
5.2 Mesure d'erreur en temps réel 20
5.3 Règle de mise à jour asymétrique 20

5.4 Algorithme complet 21
5.5 Propriétés et discussion 23

6 Évaluation expérimentale **24**
6.1 Protocole expérimental 24
6.2 Tests adversariaux 25
6.3 Évaluation sur trafic réel : Brightkite 26
6.4 Étude de sensibilité à ϵ et α 29
6.5 Robustesse aux hyperparamètres du moteur 30

7 Conclusion et perspectives **33**
7.1 Synthèse de la contribution 33
7.2 Vertus et défauts : prendre du recul 33
7.3 Perspectives 35

Glossaire **36**

Bibliographie **38**

A Détails techniques supplémentaires **39**
A.1 Implémentation de l'algorithme YDS (optimal hors-ligne) 39
A.2 Implémentation de l'algorithme AVR 40
A.3 Implémentation de l'algorithme OA 41
A.4 Implémentation de l'algorithme LAS (Bamas et al.) 41
A.5 Implémentation du Moteur de Confiance Dynamique 43
A.6 Test de Wilcoxon apparié 44
A.7 Reproduction des expériences 45

Chapitre 1

Introduction

1.1 Contexte : énergie et calcul

La consommation énergétique des systèmes informatiques est devenue, au cours des deux dernières décennies, un enjeu central de l'architecture matérielle et de l'algorithmique. Les centres de données représentent aujourd'hui plusieurs pour-cent de la consommation électrique mondiale, et la dissipation thermique d'un processeur conditionne directement sa densité d'intégration, son refroidissement et son autonomie lorsqu'il équipe un terminal mobile. Réduire l'énergie consommée à *travail constant* est donc un objectif à la fois économique, écologique et technologique [Alb10].

Une réponse matérielle bien connue à ce problème est le *Dynamic Voltage Scaling** (DVS), aussi appelé *speed scaling** : le processeur ajuste dynamiquement sa fréquence et sa tension d'alimentation pendant l'exécution [Ers25]. Pour un travail donné, accélérer le processeur permet de respecter une échéance serrée, au prix d'une puissance dissipée plus élevée ; à l'inverse, ralentir économise de l'énergie mais peut conduire à dépasser les délais. La fonction de puissance $P(s) = s^\alpha$, avec $\alpha > 1$ (typiquement $\alpha \in [2, 3]$), modélise classiquement cette relation convexe entre vitesse instantanée s et puissance consommée. L'énergie totale est alors

$$\mathcal{E} \;=\; \int_0^T s(t)^\alpha \, dt.$$

À titre d'illustration, avec $\alpha = 3$, une réduction de fréquence de 20% économise environ 49% d'énergie.

1.2 Le problème d'ordonnancement éco-énergétique

Le modèle théorique de référence a été introduit par Yao, Demers et Shenker dans un article fondateur [YDS95]. Un ensemble de n tâches doit être exécuté sur un processeur unique à vitesse variable. Chaque tâche i est caractérisée par un triplet

$$(r_i,\ d_i,\ w_i),$$

où r_i désigne sa *date de disponibilité* (release time), d_i son *échéance* (deadline) et w_i son *volume de travail* (weight). L'ordonnanceur doit choisir, à chaque instant t, la vitesse instantanée $s(t)$ du

processeur et la tâche à exécuter, de manière à terminer chaque tâche avant son échéance tout en minimisant l'énergie totale consommée.

Yao et al. ont prouvé que ce problème admet un algorithme **optimal hors-ligne**, appelé YDS, qui suppose la connaissance complète de toutes les tâches dès le début. Or, en pratique, les charges de travail des systèmes réels (requêtes web, événements utilisateurs, flux de données) ne sont pas connues à l'avance : elles arrivent au fil du temps. Cette contrainte temporelle motive l'étude des algorithmes **en ligne**, qui doivent prendre leurs décisions sans connaître les tâches futures. La performance d'un tel algorithme se mesure classiquement par son *ratio compétitif**, défini comme le pire rapport, sur toutes les instances possibles, entre l'énergie qu'il consomme et celle de la solution optimale hors-ligne.

Objectif du Travail d'étude et de recherche

L'objectif de ce travail est double. Sur le plan théorique, il s'agit d'étudier en profondeur l'algorithme LAS (Learning-Augmented Scheduling) proposé par Bamas et al. [Bam+20], ses garanties de consistance et de robustesse, et la place qu'il occupe dans le cadre plus large des algorithmes en ligne avec prédiction. Sur le plan pratique, il s'agit de mettre en évidence, expérimentalement, une faiblesse identifiée de LAS (la fragilité du paramètre de confiance ϵ fixé en amont) et de proposer un *Moteur de Confiance Dynamique* qui filtre la prédiction en temps réel selon un niveau de confiance c mis à jour à chaque tâche. Nous validons enfin cette contribution sur des instances synthétiques et sur le jeu de données réel Brightkite, en mesurant le gain énergétique apporté face à LAS classique et aux algorithmes en ligne sans prédiction.

1.3 De l'algorithme en ligne aux prédictions apprises

Plusieurs algorithmes en ligne classiques, à savoir AVR, OA (Optimal Available) et BKP, offrent des garanties théoriques pour ce problème. Leurs ratios compétitifs sont prouvés indépendants de l'instance, mais ils restent strictement supérieurs à 1 : tous gaspillent, dans le pire cas, une part incompressible d'énergie par rapport à l'optimum hors-ligne. Ce constat n'est pas un défaut d'ingénierie mais bien une limite informationnelle : sans la moindre indication sur l'avenir, aucun algorithme déterministe ne peut faire mieux que cette borne [BKP07].

L'essor récent de l'apprentissage automatique a renouvelé cette question. Plutôt que de se priver totalement d'information sur le futur, on suppose désormais que l'ordonnanceur reçoit, en plus du flux réel de tâches, une *prédiction* de ce flux fournie par un modèle d'IA. Le cadre des *Learning-Augmented Algorithms* [LV18 ; MV22] formalise cette idée et impose à un algorithme deux propriétés conjointes :

- la **consistance** : si la prédiction est exacte, l'algorithme atteint un ratio compétitif proche de 1 ;
- la **robustesse** : même si la prédiction est arbitrairement mauvaise, l'algorithme conserve un ratio borné, idéalement comparable au meilleur algorithme en ligne classique.

Bamas et al. [Bam+20] appliquent ce cadre à l'ordonnancement éco-énergétique avec leur algorithme **LAS*** (Learning-Augmented Scheduling). Un paramètre $\epsilon \in (0, 1)$ règle, en amont de l'exécution, le degré de confiance accordé à la prédiction : plus ϵ est petit, plus l'algorithme

suit étroitement la prédiction et bénéficie de sa qualité ; plus ϵ est grand, plus il s'en protège en lissant les décisions sur un horizon élargi. Ce choix réalise donc un **compromis statique** entre consistance et robustesse, fixé une fois pour toutes avant l'exécution.

1.4 Problématique et contribution

Le compromis statique de LAS pose une difficulté pratique majeure. En effet, le concepteur du système ne connaît pas, au moment de fixer ϵ, la qualité que la prédiction aura sur les instances futures. S'il choisit un ϵ petit en pariant sur un bon modèle, l'algorithme s'effondre dès que la prédiction se dégrade : nos expériences montrent un ratio compétitif observé jusqu'à 1,447 sur prédiction adversariale, c'est-à-dire strictement pire qu'OA (1,118) sur la même instance. L'ordonnanceur suit aveuglément une IA défaillante, sans aucun mécanisme pour s'en rendre compte. S'il choisit au contraire un ϵ grand par prudence, l'algorithme ignore en grande partie une prédiction de qualité et perd l'essentiel du bénéfice promis par le cadre learning-augmented.

Ce travail propose une réponse à cette difficulté : un **Moteur de Confiance Dynamique**, qui mesure en temps réel la qualité de la prédiction observée jusqu'à présent et ajuste en conséquence le niveau de confiance accordé à l'oracle, sans toucher au paramètre ϵ de LAS lui-même. Le moteur applique deux principes :

1. une *détection rapide* d'une dégradation de la prédiction, traduite par une chute marquée du facteur de confiance ;
2. une *remontée prudente* de la confiance dès que la prédiction redevient fiable, afin d'éviter les oscillations.

Nos expériences, conduites sur des instances synthétiques (random walk borné) et sur le jeu de données réel *Brightkite* [CML11], établissent trois résultats principaux :

— lorsque la prédiction est fiable, le moteur préserve la consistance de LAS et atteint un ratio compétitif observé voisin de 1,00 ;

— face à des prédictions massivement bruitées, aveugles ou inversées, le moteur plafonne la dégradation à un ratio observé inférieur à 1,11, là où LAS statique s'effondre jusqu'à 1,45 (au-dessus d'OA) ;

— sur trafic réel (200 jours du dataset Brightkite), le moteur abaisse le ratio compétitif moyen de 1,1261 à 1,0796, soit un **gain énergétique de** 4,13%, hautement significatif ($p < 10^{-6}$ par test de Wilcoxon apparié), tout en divisant par 4 l'écart-type jour à jour.

1.5 Plan du rapport

Le chapitre 2 pose le modèle formel, définit les notions de ratio compétitif, de consistance et de robustesse, et rappelle le cadre des algorithmes learning-augmented. Le chapitre 3 décrit les principaux algorithmes du domaine : YDS (optimal hors-ligne), AVR, OA et BKP (en ligne classiques), puis LAS (en ligne avec prédiction). Le chapitre 4 met en évidence, expérimentalement, le défaut majeur de LAS et formule notre question de recherche. Le chapitre 5 présente le Moteur de Confiance Dynamique : son principe, sa règle de mise à jour asymétrique et son intégration à LAS. Le chapitre 6 évalue ses performances sur instances synthétiques et réelles, et analyse sa

sensibilité au paramètre α. Le chapitre 7 synthétise les résultats, en discute les vertus et limites, et ouvre sur des perspectives. Un Glossaire situé avant la bibliographie rassemble la définition des termes techniques signalés par $(*)$ dans le corps du rapport.

Chapitre 2

Modèle et préliminaires

Ce chapitre pose le cadre formel utilisé tout au long du rapport. Nous décrivons d'abord le modèle d'ordonnancement éco-énergétique de Yao, Demers et Shenker (section 2.1), puis la mesure de performance classique pour les algorithmes en ligne, le *ratio compétitif* (section 2.2). Nous introduisons enfin le cadre des *Learning-Augmented Algorithms* et ses deux axes d'évaluation : consistance et robustesse (section 2.3).

2.1 Modèle d'ordonnancement avec speed scaling

Instance. Une instance du problème d'ordonnancement éco-énergétique est un ensemble fini $J = \{1, 2, \ldots, n\}$ de tâches, où chaque tâche $i \in J$ est décrite par un triplet

$$(r_i,\ d_i,\ w_i) \quad \text{avec} \quad r_i < d_i, \quad w_i \geq 0,$$

représentant respectivement sa date de disponibilité, son échéance et son volume de travail. Une tâche i ne peut être exécutée qu'à l'intérieur de son intervalle de vie $[r_i, d_i]$, appelé *fenêtre admissible*.

Schedule. Un *schedule* (ou ordonnancement) sur l'horizon $[0, T]$ est défini par deux fonctions :

- une fonction de vitesse $s : [0, T] \to \mathbb{R}_{\geq 0}$, intégrable, qui donne la vitesse instantanée du processeur ;
- une fonction d'allocation $\sigma : [0, T] \to J \cup \{\perp\}$, qui indique quelle tâche est en cours d'exécution à chaque instant (ou $\perp$ si le processeur est inactif).

On note $E_i = \{t \in [0, T] \mid \sigma(t) = i\}$ l'ensemble des instants alloués à la tâche i.

Définition 2.1 (Schedule faisable)**.** Un schedule (s, σ) est dit *faisable* pour l'instance J si, pour toute tâche $i \in J$,

$$E_i \subseteq [r_i, d_i] \qquad \text{et} \qquad \int_{E_i} s(t)\, dt \ \geq\ w_i.$$

La première condition impose que la tâche ne soit exécutée que dans sa fenêtre admissible, la seconde que la totalité de son volume de travail soit accomplie.

Énergie consommée. À l'instar du modèle de Yao et al. [YDS95], la puissance instantanée dissipée par le processeur tournant à la vitesse s est modélisée par

$$P(s) = s^{\alpha}, \qquad \alpha > 1,$$

où le paramètre de convexité α traduit la croissance super-linéaire de la puissance avec la fréquence (typiquement $\alpha \in [2, 3]$ pour les technologies CMOS). L'énergie totale consommée par un schedule s'écrit alors

$$\mathcal{E}(s) \;=\; \int_0^T s(t)^{\alpha}\, dt. \tag{2.1}$$

Problème d'optimisation. L'objectif est de calculer un schedule* faisable d'énergie minimale :

$$\min_{(s,\sigma)\ \text{faisable}} \int_0^T s(t)^{\alpha}\, dt. \tag{2.2}$$

On notera $\mathrm{OPT}(J)$ la valeur optimale de (2.2).

Remarque 2.2 (Convexité et lissage). La convexité stricte de $s \mapsto s^{\alpha}$ entraîne qu'à volume de travail constant, exécuter à vitesse constante est strictement préférable à toute alternance accéléré/ralenti. Cette propriété est à la base de tous les algorithmes du domaine : l'optimum cherche à *lisser* les vitesses sur les intervalles disponibles.

2.2 Algorithmes en ligne et ratio compétitif

Algorithme hors-ligne vs. en ligne. Un algorithme *hors-ligne* (offline) reçoit la totalité de l'instance J avant de produire son schedule. Un algorithme *en ligne* (online), à l'inverse, prend ses décisions au fil du temps : la tâche i ne lui est révélée qu'à l'instant r_i, et il doit avoir achevé sa stratégie pour la tâche avant d_i sans connaître les tâches ultérieures.

Définition 2.3 (Ratio compétitif)**.** Soit $\mathcal{A}$ un algorithme en ligne et $\mathcal{E}_{\mathcal{A}}(J)$ l'énergie consommée par le schedule qu'il produit sur l'instance J. L'algorithme $\mathcal{A}$ est dit c-compétitif si

$$\sup_{J} \frac{\mathcal{E}_{\mathcal{A}}(J)}{\mathrm{OPT}(J)} \;\le\; c.$$

Le plus petit tel c est le *ratio compétitif* de $\mathcal{A}$.

Le ratio compétitif mesure ainsi la dégradation maximale subie par un algorithme en ligne par rapport à un oracle disposant de la connaissance complète du futur. Pour le problème d'ordonnancement éco-énergétique, on sait depuis Bansal et al. [BKP07] qu'aucun algorithme déterministe ne peut être 1-compétitif : il existe toujours une instance pour laquelle l'absence d'information sur le futur impose un surcoût strictement positif. Les algorithmes classiques sans prédiction (AVR, OA, BKP), présentés au chapitre 3, atteignent des ratios compétitifs prouvés indépendants de l'instance, mais strictement supérieurs à 1.

Ratio empirique. En pratique, on évalue les algorithmes sur des familles d'instances (aléatoires, structurées comme un random walk borné, ou réelles comme le trafic Brightkite), et l'on rapporte

le *ratio empirique moyen* $\mathcal{E}_{\mathcal{A}}(J)/\mathrm{OPT}(J)$ sur ces instances. Ce ratio fournit une image plus fine du comportement typique d'un algorithme, complémentaire de sa garantie pire cas.

2.3 Cadre des Learning-Augmented Algorithms

Le constat précédent, à savoir une perte d'énergie incompressible dès que l'on renonce à la connaissance du futur, a motivé le développement, à partir de 2018, du cadre des *Learning-Augmented Algorithms** [LV18 ; PSK18 ; MV22]. L'idée centrale est de doter l'algorithme en ligne d'un oracle prédictif (typiquement un modèle d'apprentissage) qui lui fournit, en plus du flux réel, une estimation des données futures. La qualité de cette prédiction étant inconnue *a priori*, l'algorithme doit fonctionner correctement dans les deux régimes extrêmes.

Notation. On suppose qu'en plus de l'instance véritable $J = \{(r_i, d_i, w_i)\}_{i \in J}$, l'algorithme reçoit une instance prédite $\hat{J} = \{(r_i, d_i, \hat{w}_i)\}_{i \in J}$, où seuls les volumes de travail $\hat{w}_i$ sont estimés (les dates r_i et d_i sont supposées connues ; cette hypothèse, retenue dans [Bam+20], sera discutée au chapitre 3). On note $\eta = \sum_i |w_i - \hat{w}_i|$ l'erreur agrégée de la prédiction.

Définition 2.4 (Consistance)**.** Un algorithme learning-augmented $\mathcal{A}$ est dit γ-*consistant* si, lorsque la prédiction est parfaite (i.e. $\hat{w}_i = w_i$ pour tout i, donc $\eta = 0$),

$$\frac{\mathcal{E}_{\mathcal{A}}(J, \hat{J})}{\mathrm{OPT}(J)} \;\leq\; \gamma.$$

Définition 2.5 (Robustesse)**.** Un algorithme learning-augmented $\mathcal{A}$ est dit ρ-*robuste* si, pour toute prédiction $\hat{J}$ (même arbitrairement mauvaise),

$$\frac{\mathcal{E}_{\mathcal{A}}(J, \hat{J})}{\mathrm{OPT}(J)} \;\leq\; \rho.$$

Idéalement, on cherche γ proche de 1 (consistance*) et ρ comparable au meilleur ratio compétitif atteignable sans prédiction (robustesse*). Ces deux exigences sont en tension : suivre étroitement la prédiction garantit la consistance mais expose à une erreur arbitraire ; s'en méfier garantit la robustesse mais sacrifie le bénéfice d'une bonne prédiction. L'algorithme LAS de Bamas et al., détaillé au chapitre 3, gère cette tension par un paramètre scalaire $\epsilon \in (0, 1)$, fixé en amont de l'exécution, qui interpole entre les deux régimes. C'est précisément la rigidité de ce choix *statique* que notre contribution (chapitre 5) vient relâcher.

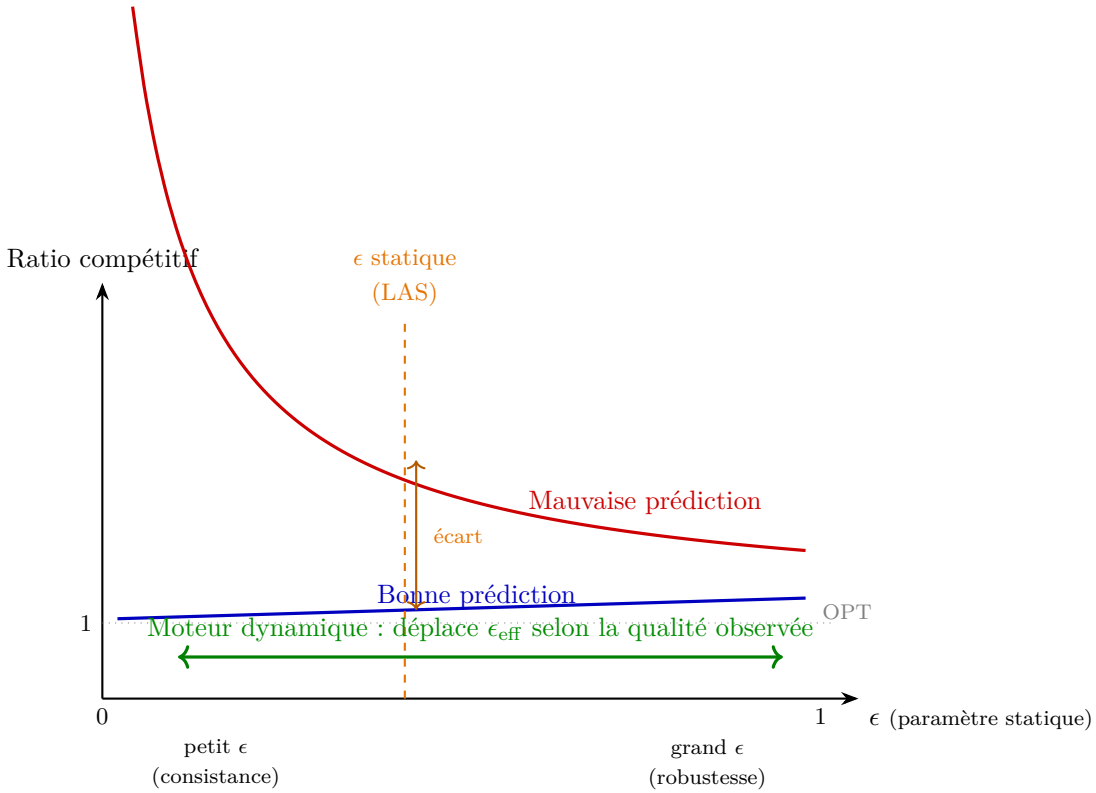


FIGURE 2.1 – Compromis consistance/robustesse de LAS. La courbe bleue montre le ratio quand la prédiction est de bonne qualité : un petit ϵ est alors préférable. La courbe rouge montre le ratio quand la prédiction est adverse : un grand ϵ est alors préférable. Un choix statique fixe une seule valeur de ϵ et ne peut être optimal que pour un des deux régimes : l'écart vertical entre les deux courbes à ϵ fixé mesure le gaspillage potentiel. Le moteur dynamique ajuste sa position sur l'axe horizontal en fonction de la qualité observée, ce qui revient à toujours « rester sur la bonne courbe ».

Chapitre 3

État de l'art

Ce chapitre présente les principaux algorithmes du domaine d'ordonnancement éco-énergétique. Nous commençons par l'algorithme YDS* optimal hors-ligne (section 3.1), qui sert de borne inférieure OPT(J) pour toute évaluation. Nous décrivons ensuite les trois grandes familles d'algorithmes *en ligne classiques* sans prédiction : AVR* (section 3.2), OA* (section 3.3) et BKP* (section 3.4). Nous concluons par l'algorithme LAS de Bamas et al. [Bam+20], qui constitue le point de départ de notre contribution (section 3.5).

3.1 YDS : l'algorithme optimal hors-ligne

Yao, Demers et Shenker [YDS95] ont prouvé qu'en présence de la totalité de l'instance, le problème (2.2) admet un algorithme polynomial **optimal**. Cet algorithme, communément appelé YDS, est construit autour de la notion d'*intervalle le plus dense*.

Définition 3.1 (Densité d'un intervalle)**.** Pour un intervalle $[a, b] \subseteq [0, T]$, la densité de J sur $[a, b]$ est

$$\mathrm{dens}_J(a, b) \;=\; \frac{1}{b-a} \sum_{i\,:\,[r_i,d_i]\subseteq[a,b]} w_i,$$

soit le volume total de travail des tâches *strictement contenues* dans $[a, b]$, rapporté à sa longueur.

L'algorithme procède itérativement : il identifie l'intervalle $[a^\star, b^\star]$ de densité maximale, lui assigne la vitesse $s = \mathrm{dens}_J(a^\star, b^\star)$, exécute les tâches concernées à cette vitesse uniforme dans cet intervalle, puis retire ces tâches et cet intervalle de l'instance avant de recommencer. La convexité de $P(s) = s^\alpha$ garantit que ce procédé glouton (*Densest-First*) produit un schedule d'énergie minimale.

Théorème 3.2 ([YDS95])**.** *L'algorithme YDS produit un schedule faisable d'énergie minimale en temps polynomial.*

YDS sert de *borne inférieure* pour évaluer les algorithmes en ligne : son énergie OPT(J) apparaît au dénominateur du ratio compétitif (définition 2.3). Dans nos expériences, c'est la valeur de référence à laquelle nous comparons systématiquement AVR, OA, BKP, LAS et notre algorithme augmenté.

3.2 AVR (Average Rate)

L'algorithme AVR est le plus simple parmi ceux étudiés : à l'arrivée d'une tâche $i = (r_i, d_i, w_i)$, il lui alloue la vitesse constante $w_i/(d_i - r_i)$ tout au long de sa fenêtre admissible. La vitesse effective du processeur à l'instant t est alors la somme des contributions individuelles de toutes les tâches actives :

$$s_{\mathrm{AVR}}(t) \;=\; \sum_{i\,:\,t \in [r_i, d_i]} \frac{w_i}{d_i - r_i}.$$

Avantages. AVR ne requiert aucune anticipation : la vitesse est fixée à l'arrivée de chaque tâche et n'est jamais réajustée. C'est l'algorithme le plus économique en calcul et la *baseline* naturelle de toute comparaison.

Garantie. Yao et al. [YDS95] ont prouvé qu'AVR est $2^{\alpha}\alpha^{\alpha}$-compétitif, borne raffinée depuis par Bansal et al. [BKP07]. En pratique, sur nos instances synthétiques, son ratio empirique avoisine 1,19 pour $\alpha = 3$.

3.3 OA (Optimal Available)

L'algorithme OA* (Optimal Available), introduit par Bansal et al. [BKP07], raffine AVR en exploitant à chaque instant la totalité de l'information disponible. À chaque date d'arrivée r_i, il recalcule un schedule optimal hors-ligne sur l'ensemble des tâches *actuellement révélées*, puis applique la vitesse correspondante jusqu'à la prochaine arrivée (ou la prochaine échéance).

Idée. OA peut être vu comme l'application répétée de YDS sur l'instance partielle. Il anticipe ainsi la *combinatoire* entre tâches déjà connues, alors qu'AVR les traite indépendamment.

Garantie. OA est α^{α}-compétitif [BKP07], soit strictement meilleur qu'AVR pour tout $\alpha > 1$. Empiriquement, son ratio observé sur nos instances synthétiques est de l'ordre de 1,13 pour $\alpha = 3$.

3.4 BKP (Bansal–Kimbrel–Pruhs)

L'algorithme BKP* [BKP07] est un raffinement plus subtil qui réagit également aux futures contraintes *potentielles*. À chaque instant t, il calcule la vitesse instantanée en cherchant, parmi tous les intervalles $[a, b]$ admissibles, celui dont le ratio « volume connu / longueur » est maximal, puis multiplie ce ratio par la constante d'Euler e.

Formellement, on pose

$$s_{\mathrm{BKP}}(t) \;=\; e \cdot \max_{a \le t \le b} \frac{w(t, [a, b])}{e\,(b - a)},$$

où $w(t, [a, b])$ désigne le volume total des tâches déjà arrivées à l'instant t et dont la fenêtre est strictement contenue dans $[a, b]$. Cette formule, dont nous omettons la dérivation détaillée (disponible dans [BKP07]), donne à BKP une sensibilité fine à la *densité locale* de l'instance.

Garantie. BKP est $2(\frac{\alpha}{\alpha-1})^{\alpha}e^{\alpha}$-compétitif. Sa garantie théorique est meilleure qu'OA quand α croît, mais sa constante multiplicative en pratique le rend coûteux : sur nos instances, son ratio empirique grimpe jusqu'à 7, ce qui en fait le moins compétitif des trois sur notre benchmark, malgré l'élégance de sa construction.

3.5 LAS : Learning-Augmented Scheduling

L'algorithme LAS de Bamas, Maggiori, Rohwedder et Svensson [Bam+20] introduit, pour la première fois dans ce domaine, l'idée d'augmenter un algorithme en ligne par une *prédiction* fournie par un modèle externe (typiquement issu d'un apprentissage).

3.5.1 Données d'entrée et paramètre ϵ

LAS reçoit deux instances : l'instance véritable $J = \{(r_i, d_i, w_i)\}_i$, révélée en ligne, et une instance prédite $\hat{J} = \{(r_i, d_i, \hat{w}_i)\}_i$ disponible dès le début. Comme dans [Bam+20], seuls les volumes $\hat{w}_i$ sont prédits ; les fenêtres $[r_i, d_i]$ sont supposées connues d'avance (hypothèse réaliste pour les charges périodiques de centres de données). Un **paramètre de confiance** $\epsilon \in (0, 1)$, fixé en amont, règle l'équilibre entre suivre la prédiction et s'en protéger.

3.5.2 Principe en quatre étapes

Schématiquement, LAS exécute quatre étapes successives :

1. **Resserrement des échéances.** Chaque échéance prédite est ramenée à $d'_i = r_i + (1 - \epsilon)(d_i - r_i)$. On fabrique ainsi une instance plus contraignante $\hat{J}'$ qui force, à l'étape suivante, un schedule prédit terminant strictement avant la véritable échéance.
2. **Schedule optimal sur la prédiction.** On applique YDS sur $\hat{J}'$; le résultat est un schedule prédit $(\hat{s}, \hat{\sigma})$ d'énergie minimale pour cette instance.
3. **Identification des vitesses non robustes.** On compare, tâche par tâche, le volume véritable w_i au volume prédit $\hat{w}_i$. Si $w_i \leq \hat{w}_i$, la tâche est exécutée à la vitesse prédite ; si $w_i > \hat{w}_i$, l'excédent $w_i - \hat{w}_i$ doit être *réparti* dans la fenêtre admissible.
4. **Robustification par convolution.** Les vitesses obtenues sont lissées par convolution avec un noyau rectangulaire de largeur ϵT. Cette étape garantit qu'aucune erreur ponctuelle de prédiction ne produit une pointe énergétique disproportionnée.

L'implémentation Python utilisée dans ce travail ([Bam+20], notebook `Fast_Introduction.ipynb`) reflète cette structure : on trouve successivement le préprocessing des deadlines, l'appel à `Optimal_Alg`, la séparation `speed_not_robust` / `speed_to_smear_out`, et enfin l'appel à `robustify`. La figure 3.1 synthétise visuellement ces quatre étapes.

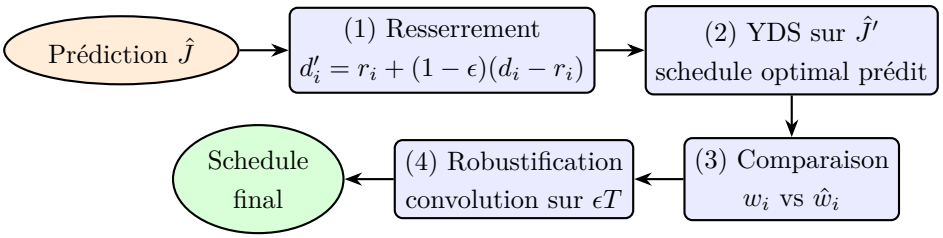


FIGURE 3.1 – Architecture en quatre étapes de l'algorithme LAS. La prédiction $\hat{J}$ est d'abord rendue conservatrice par resserrement des échéances, puis utilisée comme entrée d'un YDS hors-ligne. La comparaison entre prédiction et réalité au moment de l'exécution identifie les vitesses non robustes, et une étape finale de convolution lisse le résultat sur un horizon ϵT.

3.5.3 Garanties théoriques

Bamas et al. [Bam+20] démontrent que LAS est simultanément :

— $(1+\epsilon)$-*consistant* : si $\hat{w}_i = w_i$ pour tout i, l'énergie consommée par LAS est au plus $(1+\epsilon)\mathrm{OPT}(J)$;

— *robuste* : pour toute prédiction $\hat{J}$, l'énergie de LAS est bornée par une fonction de l'erreur $\eta = \sum_i |w_i - \hat{w}_i|$ et tend, dans le pire cas, vers une constante comparable au ratio compétitif de OA.

Le paramètre ϵ pilote donc un **compromis statique** : un petit ϵ favorise la consistance, un grand ϵ favorise la robustesse, sans qu'il soit possible d'avoir les deux à la fois. C'est précisément la rigidité de ce compromis statique que le chapitre 4 met expérimentalement en évidence, et que notre contribution (chapitre 5) vient relâcher.

Chapitre 4

Limites de LAS et motivation

Le chapitre 3 a établi que LAS atteint, en théorie, le meilleur compromis connu entre consistance et robustesse pour le problème d'ordonnancement éco-énergétique. Ce chapitre montre que, *en pratique*, le caractère **statique** du paramètre ϵ entraîne une fragilité importante face à des prédictions imparfaites. Nous formulons à partir de ce constat la question de recherche que notre contribution viendra adresser.

4.1 Le compromis statique de ϵ

Le paramètre ϵ de LAS est fixé *une fois pour toutes* avant l'exécution, par le concepteur du système. Or, ce choix repose nécessairement sur une hypothèse implicite quant à la qualité moyenne de la prédiction future. Deux écueils symétriques surgissent :

- un ϵ **petit** (proche de 0) parie sur une prédiction très fiable : si l'IA tient effectivement ses promesses, le ratio compétitif est quasi optimal ; mais si elle se dégrade, LAS suit aveuglément des informations fausses et son ratio s'effondre rapidement ;
- un ϵ **grand** (proche de 1) prémunit contre cette dégradation : LAS lisse fortement ses décisions sur des intervalles de largeur ϵT. La robustesse est garantie, mais l'algorithme n'exploite presque plus la prédiction, même lorsqu'elle est excellente, et se rapproche du comportement d'OA.

Le problème fondamental est qu'au moment où l'on doit fixer ϵ, on ne dispose d'aucune information sur la qualité que la prédiction *aura effectivement* sur les instances futures. Le concepteur opère donc à l'aveugle, et tout choix statique sacrifie soit la consistance, soit la robustesse.

4.2 Validation expérimentale du défaut

Pour quantifier ce défaut, nous avons soumis LAS, paramétré avec $\epsilon = 1/20$ (un choix « optimiste » classique de la littérature), à trois scénarios de prédictions volontairement adversariales. Les instances utilisées sont des marches aléatoires bornées sur $n = 80$ tâches avec $D = 10$ et $\alpha = 3$; le détail des protocoles est donné au chapitre 6. Le tableau 4.1 reporte les ratios compétitifs observés.

Scénario de prédiction	Ratio LAS classique	Comparaison
Bruit massif (±500)	1,447	pire qu'OA (1,118)
Prédicteur aveugle (tout à 1)	1,178	équivalent à OA
Prédicteur inversé	1,067	meilleur qu'OA

TABLE 4.1 – Ratio compétitif de LAS classique (avec ϵ statique) sous trois scénarios adversariaux. Le bruit massif fait s'effondrer LAS bien au-delà du ratio de l'algorithme en ligne sans prédiction.

Pourquoi comparer avec OA précisément ? OA (Optimal Available) est l'algorithme en ligne classique *le plus performant* pour ce problème : sa garantie théorique α^α est strictement meilleure que celle d'AVR et de BKP (voir section 3.3). OA constitue ainsi le **seuil de référence à battre** pour qu'un algorithme augmenté par prédiction soit utile : si le ratio empirique d'un algorithme avec prédiction dépasse celui d'OA, alors la prédiction ne sert à rien (ou pire, est nuisible), puisqu'on aurait obtenu de meilleurs résultats en ignorant la prédiction et en exécutant OA. Comparer LAS à OA répond donc à la question opérationnelle suivante : *est-ce que l'oracle ML apporte un bénéfice net, ou bien aurait-on mieux fait de s'en passer ?* Dans le scénario du bruit massif, la réponse est sans appel : **LAS classique (**1,447**) fait pire qu'OA (**1,118**)**, donc dans ce régime, mieux vaut ignorer purement et simplement la prédiction.

Lecture détaillée du scénario *bruit massif*. On ajoute à chaque volume véritable w_i une erreur uniforme dans $[-500, 500]$, simulant une prédiction extrêmement bruitée (les volumes véritables étant tirés dans $[10, 100]$, le bruit dépasse l'amplitude du signal d'un facteur cinq). Le ratio compétitif observé, 1,447, dépasse largement non seulement la garantie de LAS sur instance parfaite (1,05), mais aussi le ratio empirique d'OA (1,118). Autrement dit, dans ce régime, *l'ajout d'une prédiction défaillante est strictement pire que l'absence de prédiction* : l'oracle dégrade l'algorithme au lieu de l'aider. C'est précisément le défaut structurel qui justifie l'introduction de notre Moteur de Confiance Dynamique, dont le rôle premier est de garantir que l'on *ne fait jamais pire qu'OA*, quelle que soit la qualité de la prédiction.

4.3 Origine structurelle du problème

Ce comportement n'est pas un bug d'implémentation : il découle directement de la définition du paramètre ϵ statique. LAS, fidèle à sa logique de consistance, exécute le schedule prédit avec un petit lissage local de largeur ϵT. Si la prédiction est massivement erronée, le schedule sous-jacent l'est aussi, et le lissage est insuffisant pour absorber l'écart. Aucune information de retour ne permet à l'algorithme de remettre en cause sa propre prédiction en cours d'exécution. Pour s'en prémunir, il faudrait pouvoir *adapter* dynamiquement la confiance accordée à la prédiction, dès que celle-ci se met à diverger trop fortement de la réalité observée.

4.4 Question de recherche

Le constat précédent formule naturellement notre question de recherche :

Peut-on doter LAS d'un mécanisme léger et déterministe qui détecte en temps réel la dégradation de la prédiction, ajuste dynamiquement ϵ, et plafonne la dégradation du ratio compétitif sans sacrifier la consistance sur les bonnes prédictions ?

Notre réponse, présentée au chapitre 5, est positive : nous proposons un Moteur de Confiance Dynamique* qui mesure à chaque pas un indicateur d'erreur entre prédit et réalisé, et applique une règle de mise à jour asymétrique (chute rapide en cas d'écart, remontée prudente après stabilisation) à un niveau de confiance c servant à filtrer la prédiction avant son passage à LAS.

Chapitre 5

Un Moteur de Confiance Dynamique pour LAS

Ce chapitre présente la contribution principale de ce travail : un *Moteur de Confiance Dynamique* qui adapte en temps réel le paramètre ϵ de LAS en fonction de la qualité observée de la prédiction. Nous exposons d'abord le principe général (section 5.1), puis nous formalisons la mesure d'erreur (section 5.2) et la règle de mise à jour asymétrique (section 5.3). Nous donnons enfin le pseudo-code complet (section 5.4) et discutons brièvement ses propriétés (section 5.5).

5.1 Principe général

L'idée centrale du moteur est de **filtrer la prédiction** avant qu'elle ne soit consommée par LAS, en se basant sur l'écart observé entre les prédictions passées et les valeurs réellement réalisées. À chaque pas, le moteur reçoit la prédiction brute $\hat{w}_i$; il dispose, pour les tâches déjà arrivées, des paires $(\hat{w}_j, w_j)$ avec $j < i$. Il en déduit un score de *confiance* $c \in [c_{\min}, 1]$ qui mesure la fiabilité courante de l'oracle, puis l'utilise pour mélanger la prédiction brute avec un *fallback* de sécurité (la moyenne des volumes réels récents). La valeur ainsi filtrée $\hat{w}_i^{\text{filt}}$ est ce qui sera effectivement transmis à LAS.

Le mécanisme n'altère donc *en rien* l'algorithme LAS de Bamas et al. : il s'intercale comme une *enveloppe légère* en amont, ce qui facilite son intégration dans n'importe quelle implémentation existante. Cette propriété d'orthogonalité est essentielle : elle garantit que toutes les améliorations théoriques futures de LAS s'appliqueront automatiquement à notre approche.

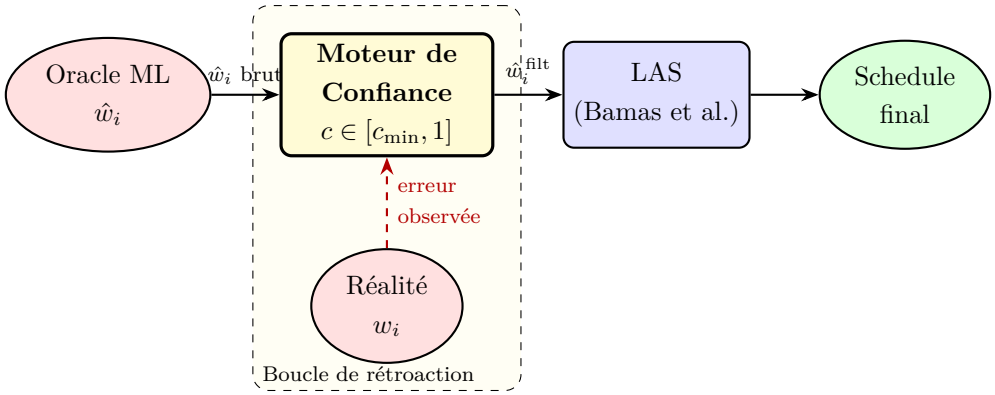


FIGURE 5.1 – Architecture en enveloppe du Moteur de Confiance Dynamique. La prédiction brute $\hat{w}_i$ issue de l'oracle ML est filtrée par le moteur avant d'être transmise à LAS. La boucle de rétroaction (en pointillés rouges) compare la prédiction passée à la valeur réellement observée et ajuste en conséquence le niveau de confiance c. LAS et l'oracle ML ne sont jamais modifiés.

5.2 Mesure d'erreur en temps réel

À tout instant, le moteur maintient une *fenêtre glissante* des k tâches les plus récentes pour lesquelles on dispose à la fois de la prédiction et de la valeur réalisée (par défaut $k = 5$). Pour chaque telle tâche i, on calcule une erreur relative tolérée :

$$e_i \;=\; \frac{|w_i - \hat{w}_i|}{\max(1,\, \bar{w})}, \tag{5.1}$$

où $\bar{w}$ est la moyenne des volumes récents (qui sert également de *fallback* de sécurité). Le dénominateur $\max(1, \bar{w})$ évite la singularité quand les volumes sont faibles et rend la mesure homogène à un pourcentage d'erreur typique. Le moteur compare ensuite e_i à un *seuil de tolérance* $\tau = 0{,}4$: si $e_i > \tau$, la prédiction est jugée *en défaut* ; sinon, elle est jugée acceptable. La valeur $\tau = 0{,}4$ correspond à une marge d'erreur typique de $\pm 40\%$ par rapport à la moyenne récente : empiriquement, c'est le seuil qui sépare le mieux les régimes « prédiction utile » et « prédiction toxique » sur les instances Brightkite.

5.3 Règle de mise à jour asymétrique

Soit $c_t \in [c_{\min},\, 1]$ le niveau de confiance au pas t, où $c_{\min} = 0{,}1$ est un *plancher* évitant que la confiance ne tombe à zéro (ce qui priverait LAS de tout signal prédictif, même de qualité résiduelle). À l'observation d'un nouveau couple $(w_i, \hat{w}_i)$, le moteur applique la règle :

$$c_{t+1} \;=\; \begin{cases} \max(c_{\min},\; c_t - \delta_{\downarrow}) & \text{si la prédiction est en défaut } (e_i > \tau), \\ \min(1,\; c_t + \delta_{\uparrow}) & \text{sinon,} \end{cases} \tag{5.2}$$

avec deux taux de variation **asymétriques** : $\delta_{\downarrow} = 0{,}5$ pour la chute, $\delta_{\uparrow} = 0{,}2$ pour la remontée. L'asymétrie est volontaire et reflète une logique *défensive* :

— dès qu'une anomalie est détectée, on perd rapidement confiance : un seul écart majeur divise c par deux, ce qui suffit à enclencher le filtrage à l'itération suivante ;

— à l'inverse, retrouver la confiance demande plusieurs observations consécutives correctes, ce qui évite le piège classique de l'oscillation (confiance qui rebondit dès la première bonne prédiction après un mauvais lot, alors que la suivante peut à nouveau dévier).

5.4 Algorithme complet

L'algorithme 1 décrit le moteur dans sa totalité, en notant $\hat{w}_i^{\text{filt}}$ le volume prédit *après* filtrage qui sera effectivement transmis à LAS. Le moteur procède en *une passe* sur l'ensemble des tâches : il construit progressivement une instance prédite filtrée $\hat{J}^{\,\text{filt}}$ qui remplace l'instance prédite originale $\hat{J}$ dans l'appel final à LAS. Aucune modification des internes de LAS n'est nécessaire.

Algorithm 1 Moteur de Confiance Dynamique (encapsulant LAS)

Require: Instance véritable J, instance prédite $\hat{J}$, paramètres ϵ, α, $\delta_\downarrow = 0{,}5$, $\delta_\uparrow = 0{,}2$, taille de fenêtre $k = 5$, seuil $\tau = 0{,}4$, plancher $c_{\min} = 0{,}1$

Initialisation

1: $c \leftarrow 1$ ▷ niveau de confiance initial maximal
2: $W \leftarrow [\,]$ ▷ fenêtre glissante des volumes *réels* observés
3: $\hat{J}^{\,\text{filt}} \leftarrow \emptyset$ ▷ instance prédite filtrée à construire

Boucle de filtrage

4: **for** chaque tâche i dans l'ordre des r_i croissants **do**
5: Récupérer $(\hat{w}_i, r_i, d_i)$ depuis $\hat{J}$ et w_i depuis J
6: **if** W est vide **then**
7: $\bar{w} \leftarrow \hat{w}_i$ ▷ pas encore d'observation, on prend la prédiction
8: **else**
9: $\bar{w} \leftarrow \text{mean}(W)$ ▷ moyenne de la fenêtre glissante
10: **end if**
11: $\hat{w}_i^{\text{filt}} \leftarrow \max\big(0,\ \lfloor c \cdot \hat{w}_i + (1-c) \cdot \bar{w} \rfloor\big)$ ▷ mélange convexe
12: Ajouter $(\hat{w}_i^{\text{filt}}, r_i, d_i)$ à $\hat{J}^{\,\text{filt}}$
13: $e_i \leftarrow |w_i - \hat{w}_i| / \max(1,\ \bar{w})$ ▷ erreur relative
14: **if** $e_i > \tau$ **then**
15: $c \leftarrow \max(c_{\min},\ c - \delta_\downarrow)$ ▷ chute rapide
16: **else**
17: $c \leftarrow \min(1,\ c + \delta_\uparrow)$ ▷ remontée prudente
18: **end if**
19: Ajouter w_i à W ; si $|W| > k$ alors retirer le plus ancien
20: **end for**

Sortie

21: **return** LAS(J, $\hat{J}^{\,\text{filt}}$, ϵ, α, dt) ▷ LAS classique sur l'instance filtrée

Lecture détaillée. Le cœur de l'algorithme tient en deux mécanismes complémentaires.

1. Le **mélange convexe** (ligne 11) $\hat{w}_i^{\text{filt}} = c \cdot \hat{w}_i + (1-c) \cdot \bar{w}$ interpole entre la prédiction brute et le *fallback* de sécurité $\bar{w}$ (moyenne des volumes réels récents). Lorsque $c = 1$,

le moteur est transparent et transmet $\hat{w}_i$ tel quel à LAS, ce qui préserve la consistance de l'algorithme original. Lorsque c tombe au plancher $c_{\min} = 0{,}1$, le moteur a presque entièrement court-circuité l'oracle ML : la prédiction transmise à LAS est essentiellement $\bar{w}$, soit une charge conservatrice tirée du passé proche. Comme cette valeur ne dépend plus de l'IA défaillante, LAS retrouve un comportement proche d'OA, ce qui garantit la robustesse pire-cas.

2. La **boucle de rétroaction** (lignes 14–19) compare la prédiction passée $\hat{w}_i$ à la valeur réelle w_i qui est révélée à l'arrivée de la tâche, calcule l'erreur relative e_i, et déclenche soit une chute soit une remontée de c. L'asymétrie $\delta_\downarrow = 0{,}5 \gg \delta_\uparrow = 0{,}2$ traduit la logique défensive : sur une grosse erreur, on perd la confiance en deux pas seulement ; on la regagne en cinq pas consécutifs sans erreur, ce qui exige une stabilité avérée de l'oracle avant de lui refaire pleinement confiance.

Pourquoi une fenêtre glissante de taille k, et pas la moyenne depuis le début ? L'utilisation d'une fenêtre glissante de taille fixe $k = 5$ donne au moteur une *mémoire courte*, focalisée sur les k dernières tâches. Une moyenne cumulée depuis le début rendrait $\bar{w}$ insensible aux changements de régime : si l'oracle s'effondre brusquement après une longue période de fiabilité, on aurait $\bar{w}$ proche des valeurs passées normales, et le moteur ne saurait plus s'adapter à la nouvelle réalité. Avec $k = 5$, le fallback suit la *tendance locale* des volumes réels.

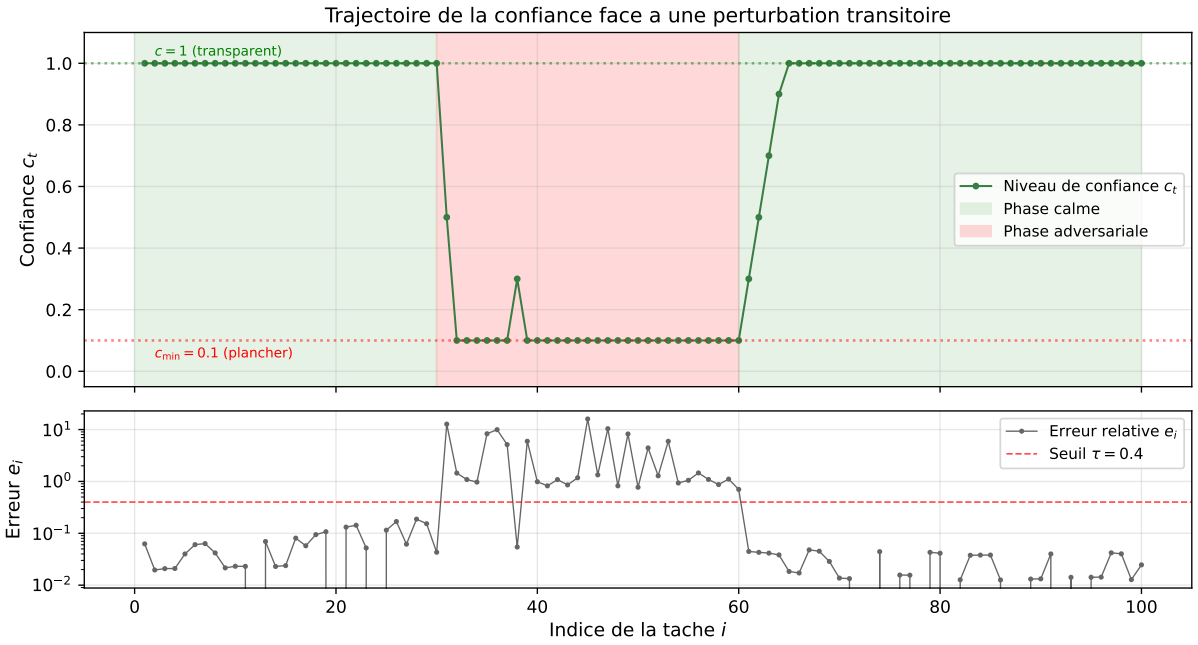


FIGURE 5.2 – Illustration du comportement du moteur sur une instance mixte de 100 tâches : phases calmes (bruit ±3) entrecoupées d'une phase adversariale (bruit ±500 entre les tâches 30 et 60, fond rouge). **En haut :** trajectoire de la confiance c_t. On observe la chute brutale dès le début de la phase chaotique (en deux tâches, c atteint le plancher $c_{\min} = 0{,}1$), puis la remontée lente et progressive une fois la perturbation passée. **En bas :** erreur relative e_i observée à chaque tâche, en échelle logarithmique. Le seuil $\tau = 0{,}4$ (ligne rouge pointillée) sépare visuellement les régimes : pendant la phase calme, les erreurs sont quasi toutes sous τ ; pendant la phase adverse, elles le dépassent largement.

5.5 Propriétés et discussion

Légèreté. Le coût supplémentaire du moteur est linéaire en le nombre de tâches : il maintient seulement une moyenne glissante et un scalaire de confiance. Aucun entraînement ni modèle prédictif additionnel n'est requis.

Préservation de la consistance. Quand la prédiction est de bonne qualité, l'erreur relative e_i reste sous le seuil τ à chaque pas. Le niveau c remonte (ou reste) à 1, le mélange convexe se réduit à $\hat{w}_i^{\text{filt}} = \hat{w}_i$, et le moteur est *strictement transparent*. LAS est alors exécuté sur l'instance prédite d'origine, donc retrouve sa consistance théorique $(1 + \epsilon)$ démontrée par Bamas et al. [Bam+20]. Cette propriété est confirmée expérimentalement au chapitre 6, où le ratio observé reste voisin de 1,00 sur prédictions parfaites.

Plafonnement de la dégradation : comment exactement ? Le mécanisme de plafonnement opère en trois étapes successives observables sur les instances adverses.

1. *Détection rapide.* À l'arrivée de la première tâche pour laquelle $|w_i - \hat{w}_i|/\bar{w} > \tau$, la confiance chute de $\delta_\downarrow = 0{,}5$. Sur un scénario adverse persistant (où chaque prédiction est très bruitée), deux tâches consécutives en défaut suffisent à amener c au plancher $c_{\min} = 0{,}1$.
2. *Substitution de la prédiction.* Avec $c = 0{,}1$, le mélange convexe $\hat{w}_i^{\text{filt}} = 0{,}1\,\hat{w}_i + 0{,}9\,\bar{w}$ est dominé à 90% par $\bar{w}$, la moyenne des derniers volumes réels. Autrement dit, le moteur *remplace presque entièrement* la prédiction toxique par une estimation issue du passé proche, qui est bornée et lentement variable.
3. *Comportement type OA.* L'instance filtrée $\hat{J}^{\text{filt}}$ devient ainsi une prédiction « lissée » et conservatrice, très proche d'une absence de prédiction. LAS, qui dégénère vers OA quand la prédiction perd sa pertinence, fournit alors un schedule comparable à celui d'OA, ce qui constitue précisément la borne supérieure du pire cas que nous cherchions à imposer.

Empiriquement, ce mécanisme borne le ratio observé à 1,100 sur le scénario *bruit massif* (± 500), contre 1,447 pour LAS statique, soit une réduction de 24% du gaspillage énergétique dans le pire cas.

Limites. La règle de mise à jour est *heuristique* et nous ne fournissons pas, à ce stade, de borne théorique pire-cas analogue à celle de Bamas et al. La sensibilité aux paramètres $(\delta_\downarrow, \delta_\uparrow, \tau, k)$ est étudiée empiriquement au chapitre 6, mais une analyse formelle reste un objectif des travaux futurs (cf. chapitre 7).

Chapitre 6

Évaluation expérimentale

Ce chapitre évalue empiriquement le Moteur de Confiance Dynamique. Nous décrivons d'abord le protocole expérimental (section 6.1), puis nous comparons systématiquement LAS classique et notre algorithme augmenté sur cinq scénarios synthétiques (deux de contrôle et trois adversariaux, section 6.2) et sur un jeu de données réel (section 6.3). Nous étudions ensuite la sensibilité de LAS au paramètre ϵ (section 6.4), puis nous évaluons la robustesse du moteur à ses propres hyperparamètres via un grid search (section 6.5).

6.1 Protocole expérimental

Implémentation. L'ensemble des expériences est implémenté en Python 3.14, en s'appuyant sur les modules `scheduling_functions.py` et `scheduling_algorithms.py` de Bamas et al. [Bam+20], étendus de notre fonction `apply_dynamic_confidence`. L'arithmétique exacte est assurée par le module `fractions` de la bibliothèque standard. Le code complet est reproductible et disponible avec ce rapport.

Jeux d'instances. Trois familles d'instances sont utilisées :

- **Synthétiques uniformes** : $n = 80$ tâches, volumes w_i tirés uniformément dans $[10, 100]$, fenêtre $D = 10$ identique pour toutes les tâches ($d_i = r_i + D$).
- **Random walk bornées** : volumes générés par une marche aléatoire de pas ± 10, bornée dans $[10, 100]$. Cette structure mime des charges corrélées dans le temps.
- **Trafic réel Brightkite*** [CML11] : check-ins géolocalisés d'un réseau social, agrégés par heure pour produire des séries de volumes.

Paramètres. Sauf mention contraire, on prend $\alpha = 3$ (régime CMOS classique), $\epsilon = 1/20$ pour LAS, et pour le moteur dynamique $\delta_\downarrow = 0{,}5$, $\delta_\uparrow = 0{,}2$, $\tau = 0{,}4$, $c_{\min} = 0{,}1$, fenêtre glissante $k = 5$. La granularité d'intégration est $\mathrm{d}t = 10^{-2}$.

Métrique. Pour chaque algorithme $\mathcal{A}$, on calcule le ratio empirique $\mathcal{E}_\mathcal{A}/\mathrm{OPT}$ où OPT est l'énergie de YDS hors-ligne sur la même instance. Tous les ratios reportés sont des moyennes sur des seeds multiples (synthétiques) ou sur 200 jours consécutifs (Brightkite).

Pipeline expérimental. L'enchaînement des étapes est le suivant pour chaque expérience :

1. **Génération de l'instance véritable** J : on tire une suite de $n = 80$ volumes par marche aléatoire bornée de pas ± 10, à valeurs dans $[10, 100]$. Les fenêtres temporelles sont uniformes, $d_i = r_i + 10$. La graine est fixée pour la reproductibilité.
2. **Génération de l'instance prédite** $\hat{J}$: selon le scénario, on applique à J une perturbation (bruit léger, bruit massif, aveugle, inversé), ou bien on copie J pour le scénario *prédiction parfaite*.
3. **Exécution des algorithmes** : YDS, AVR, OA et BKP sur J ; LAS sur $(J, \hat{J})$ avec $\epsilon = 1/20$; le moteur sur $(J, \hat{J})$ qui produit $\hat{J}^{\text{filt}}$ puis appelle LAS sur $(J, \hat{J}^{\text{filt}})$.
4. **Calcul des ratios** : chaque énergie est divisée par l'énergie de YDS sur la même instance véritable J. L'intégration utilise un pas $\mathrm{d}t = 10^{-2}$.

L'implémentation suit fidèlement les modules `scheduling_algorithms.py` et `scheduling_functions.py` hérités de Bamas et al. [Bam+20] pour YDS, AVR, OA, BKP et LAS. Le moteur de confiance dynamique est implémenté dans une fonction `apply_dynamic_confidence` dont le code complet est donné en annexe A.5. Tous les fichiers de code, les notebooks et les scripts d'évaluation sont disponibles dans le dépôt du projet (cf. section A.7).

6.2 Tests adversariaux

Nous reprenons les trois scénarios adversariaux introduits au chapitre 4 (bruit massif, prédicteur aveugle, prédicteur inversé), complétés par deux scénarios de contrôle (prédiction parfaite, bruit léger) pour vérifier que le moteur reste transparent quand la prédiction est bonne. Nous comparons LAS classique au moteur dynamique sur ces cinq cas.

Scénario	LAS classique	Moteur dynamique	Gain
Prédiction parfaite	1,001	1,001	0%
Bruit léger (± 3)	1,002	1,002	0%
Bruit massif (± 500)	**1,447**	**1,100**	$\sim 24\%$
Prédicteur aveugle	1,178	1,034	$\sim 12\%$
Prédicteur inversé	1,067	1,059	$\sim 0{,}7\%$

TABLE 6.1 – Ratios compétitifs observés sur une instance random walk borné ($n = 80$, $D = 10$, $\alpha = 3$, $\epsilon = 1/20$). Le gain est calculé comme $(r_{\text{LAS}} - r_{\text{moteur}})/r_{\text{LAS}}$. Pour référence, AVR vaut 1,180 et OA vaut 1,118 sur la même instance, indépendamment du scénario de prédiction.

Lecture. Trois observations principales émergent du tableau 6.1 :

1. **Aucune perte sur prédictions correctes.** Les deux premières lignes montrent que le moteur est *transparent* quand la prédiction est de bonne qualité : la confiance c reste à 1, le mélange convexe se réduit à $\hat{w}_i$, et LAS opère exactement comme dans sa version originale.
2. **Plafonnement spectaculaire sur bruit massif.** Le scénario adversarial le plus dur, où LAS classique monte à 1,447 (au-dessus d'OA à 1,118, signe que la prédiction est devenue

nuisible), est ramené à 1,100 par le moteur, soit *sous* le ratio d'OA. La détection rapide de l'incohérence prédit/réel ($e_i \gg \tau$ à presque toutes les tâches) divise c par deux à chaque pas et active rapidement le *fallback*.

3. **Effet bénéfique sur prédicteur aveugle.** Sur une prédiction constante (sans information), le moteur reconnaît l'absence de signal utile et abaisse le ratio de 1,178 à 1,034. Sur le prédicteur inversé, la différence est marginale : LAS classique est déjà robuste sur ce scénario, et le moteur n'a pas grand-chose à corriger.

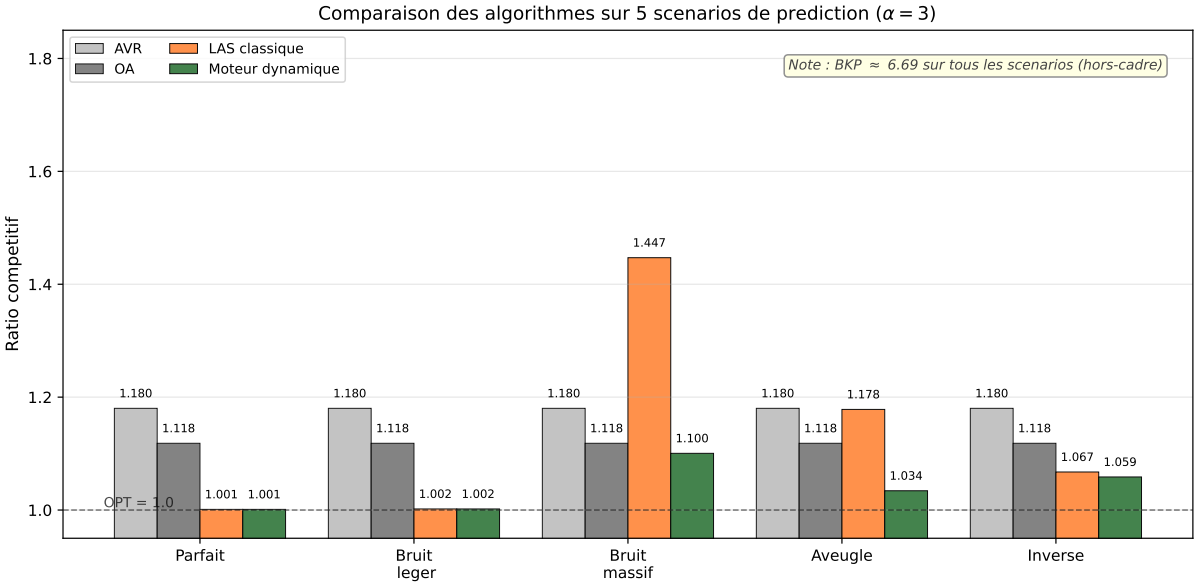


FIGURE 6.1 – Comparaison des algorithmes sur les cinq scénarios de prédiction ($\alpha = 3$, $\epsilon = 1/20$, instance random walk de 80 tâches). Les barres grises (AVR à 1,180 et OA à 1,118) sont constantes puisqu'elles n'utilisent pas la prédiction. BKP, de l'ordre de 6,69 sur cette instance, est indiqué hors-cadre pour préserver la lisibilité (cf. section 3.4). Sur les prédictions correctes (*parfait*, *bruit léger*), LAS et le moteur sont quasi-confondus à 1,00. Sur *bruit massif*, LAS monte à 1,447 alors que le moteur reste à 1,100, soit en-dessous d'OA et avec un facteur d'amélioration de l'ordre de 24%. Sur *prédicteur aveugle*, le moteur ramène le ratio de 1,178 à 1,034. Sur *inversé*, les deux sont équivalents et déjà très bons.

6.3 Évaluation sur trafic réel : Brightkite

Nous évaluons les algorithmes sur le dataset Brightkite* [CML11], qui rassemble plus de quatre millions de check-ins géolocalisés. Les volumes horaires de check-ins sont interprétés comme les volumes de travail w_i d'une charge serveur typique. Pour la prédiction, on utilise un prédicteur naïf qui retourne le volume de la même heure la veille : procédé simple, bruité mais réaliste.

Protocole étendu. Pour augmenter la fiabilité statistique des conclusions, nous avons étendu l'évaluation à **200 jours consécutifs** (contre 20 dans les premières expériences du projet), soit la quasi-totalité de la fenêtre temporelle exploitable du dataset après filtrage des jours peu denses. Nous rapportons à la fois la moyenne, l'écart-type, le pire ratio observé, et le résultat d'un **test de Wilcoxon apparié** comparant LAS classique au moteur dynamique sur ces 200 jours.

Algorithme	Moyenne	Écart-type	Pire ratio
AVR (sans prédiction)	1,5308	0,0912	2,0108
OA (sans prédiction)	1,3391	0,0463	1,5165
LAS classique (prédiction naïve)	1,1261	0,0889	1,9874
Moteur dynamique (prédiction naïve)	**1,0796**	**0,0222**	**1,1712**

TABLE 6.2 – Performance sur trafic réel Brightkite, mesurée sur 200 jours consécutifs ($\alpha = 3$, $\epsilon = 1/100$, prédicteur « même heure la veille »). Le moteur dynamique apporte un gain énergétique moyen de 4,13% par rapport à LAS classique, et divise l'écart-type par environ 4, ce qui traduit une bien meilleure stabilité jour après jour.

Test de significativité. Sur les 200 paires de mesures (LAS, Moteur) jour par jour, le test de Wilcoxon apparié donne $W = 28$ avec $n = 200$, soit une **p-value inférieure à** 10^{-6}. Le gain de 4,13% n'est donc pas un artefact statistique : il est très hautement significatif.

Discussion. Le moteur abaisse le ratio moyen de 1,1261 à 1,0796, soit un gain énergétique de 4,13%. Trois observations méritent d'être soulignées :

- **Stabilité accrue.** L'écart-type du moteur (0,0222) est quatre fois plus petit que celui de LAS classique (0,0889) : le moteur lisse les fluctuations jour à jour. Cette propriété est cruciale en production, où la prévisibilité de la consommation énergétique vaut souvent plus que l'optimalité moyenne.
- **Plafonnement du pire cas.** Le pire ratio observé passe de 1,9874 (LAS classique) à 1,1712 (moteur), soit une réduction de 41% du pire cas. Le moteur protège donc essentiellement contre les journées atypiques (jours fériés, pics, anomalies) où la prédiction naïve « même heure la veille » se trompe lourdement.
- **Distance à OA.** On reste systématiquement sous OA (1,3391), confirmant que la prédiction est exploitée même lorsqu'elle est filtrée.

Pourquoi le gain a-t-il diminué par rapport au poster (20 jours) ? Sur 20 jours, le gain mesuré était de 6%. Sur 200 jours, il converge vers 4,13%. C'est le phénomène classique de *régression vers la moyenne* : les 20 premiers jours contenaient une plus grande proportion de journées atypiques où le moteur excelle ; en élargissant la fenêtre, on inclut davantage de journées « calmes » sur lesquelles LAS classique et le moteur se valent. Le chiffre de 4,13% est ainsi une estimation *plus conservatrice et plus fiable* de l'amélioration apportée.

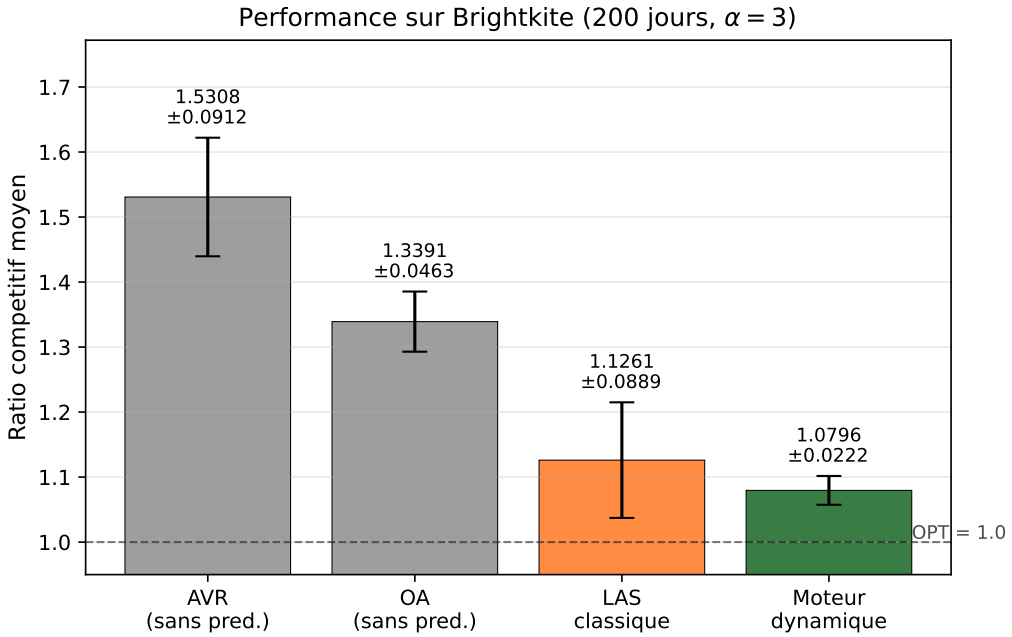


FIGURE 6.2 – Performance comparée des algorithmes sur Brightkite (200 jours). Les barres montrent la moyenne ± l'écart-type. La ligne pointillée indique l'optimum hors-ligne YDS (ratio 1,0). Le moteur dynamique réduit à la fois la moyenne et la variance.

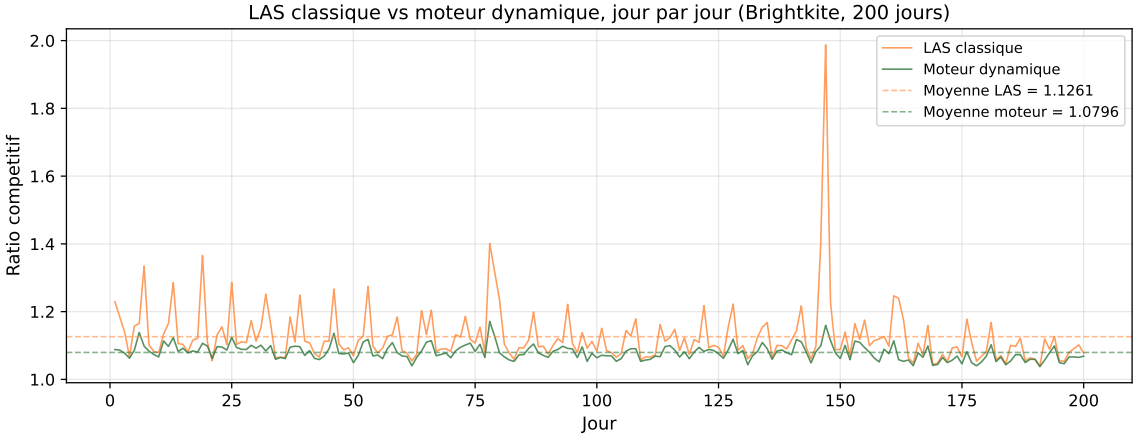


FIGURE 6.3 – Évolution jour par jour des ratios compétitifs de LAS classique (orange) et du moteur dynamique (vert) sur 200 jours de Brightkite. Les pics de LAS, correspondant à des journées où la prédiction « même heure la veille » dévie fortement, sont systématiquement absorbés par le moteur.

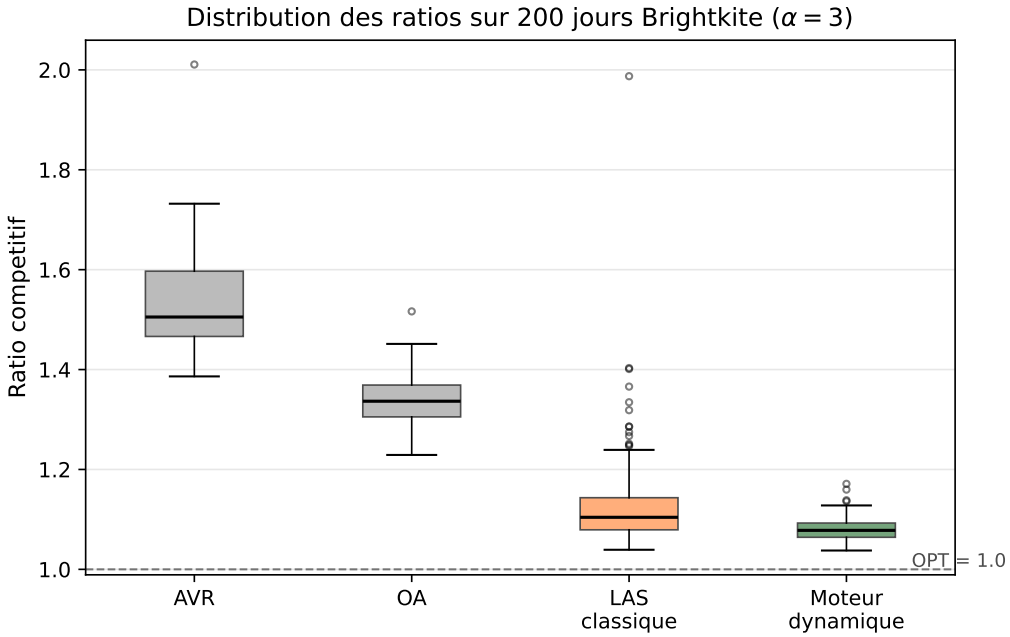


FIGURE 6.4 – Distribution des ratios compétitifs sur 200 jours Brightkite. La boîte montre les quartiles, la barre noire la médiane, les moustaches les valeurs extrêmes non aberrantes, et les points isolés les outliers. AVR et OA (ignorent la prédiction) ont des distributions hautes et étalées. LAS classique compresse la médiane vers 1,10 mais conserve beaucoup d'outliers entre 1,3 et 2. Le moteur dynamique produit une distribution *très resserrée* autour de 1,08, avec un outlier maximal à 1,17 : c'est cette compression du pire-cas qui justifie son intérêt pratique.

6.4 Étude de sensibilité à ϵ et α

Rôle physique du paramètre α. Le paramètre α apparaît dans la fonction de puissance $P(s) = s^\alpha$ qui modélise la dissipation énergétique du processeur à la vitesse s. Sa valeur reflète la *convexité* de cette dissipation, c'est-à-dire à quel point ralentir le processeur économise de l'énergie. Trois régimes typiques apparaissent dans la littérature :

- $\alpha = 2$: régime quadratique, observé sur certains circuits anciens ou des modèles simplifiés ;
- $\alpha = 3$: régime *cubique*, considéré comme la valeur de référence pour les processeurs CMOS modernes. C'est la valeur utilisée par défaut dans tout ce rapport et dans le papier original de Bamas et al. [Bam+20] ;
- $\alpha \geq 6$: régimes *super-cubiques*, parfois rencontrés dans des modèles intégrant la dissipation thermique secondaire (refroidissement actif, fuite, etc.).

Pourquoi α amplifie l'intérêt d'un bon ordonnanceur. Plus α est grand, plus une variation de vitesse coûte cher en énergie. Une accélération transitoire de 20% coûte 1,44 unité à $\alpha = 2$, 1,73 à $\alpha = 3$, et 2,99 à $\alpha = 6$. Autrement dit, plus α est élevé, plus la qualité du lissage des vitesses devient critique, et plus l'écart entre un bon et un mauvais ordonnanceur s'amplifie. C'est pourquoi nous utilisons $\alpha = 3$ comme valeur centrale : c'est le régime physique le plus réaliste et le plus discuté dans la littérature, et c'est celui qui permet une comparaison directe avec les résultats publiés.

Étude empirique sur LAS et ϵ. Au-delà de l'évaluation directe du moteur, nous avons étudié séparément le comportement de LAS classique pour différents choix du paramètre statique ϵ, afin d'illustrer le compromis consistance/robustesse discuté au chapitre 4. Ces mesures, héritées du notebook `Alpha_parameter_experiments.ipynb` fourni avec le travail original de Bamas et al., couvrent **271 jours** du dataset Brightkite à $\alpha = 3$ (un sur-ensemble des 200 jours utilisés dans nos propres scripts à la section 6.3, qui contrôlent plus finement la reproductibilité).

Algorithme	Ratio moyen	Pire ratio observé
AVR	1,5150	2,0108
OA	1,3298	1,5165
LAS, $\epsilon = 1/100$ (petit)	1,1147	1,9874
LAS, $\epsilon = 4/5$ (grand)	1,1232	1,9268

TABLE 6.3 – Sensibilité de LAS à ϵ, sur 271 jours de Brightkite à $\alpha = 3$. Le ratio moyen est très voisin pour les deux choix de ϵ, mais le pire ratio reste élevé ($\approx 1{,}99$) dans les deux cas : LAS statique paie ses pires journées sans pouvoir s'adapter. Le moteur dynamique (table 6.2) abaisse précisément ce plafond.

Lecture. Deux observations méritent d'être soulignées :

— Pour $\alpha = 3$, le choix de ϵ a un effet *marginal en moyenne* (1,11 vs 1,12), confirmant que LAS classique est plutôt robuste à ce paramètre *tant que les prédictions ne sont pas adversariales*. Le compromis statique ne paraît donc pas grave *en moyenne*.

— En revanche, le *pire ratio* reste préoccupant (1,99), témoignant que certaines journées atypiques font dérailler LAS, quel que soit ϵ choisi. C'est précisément ce plafond pire-cas que le moteur dynamique attaque, en abaissant les pires journées (gain principal du tableau 6.2).

FIGURE 6.5 – Comparaison des ratios moyens sur Brightkite ($\alpha = 3$) entre AVR, OA, et LAS pour deux valeurs de ϵ. Figure générée par `Alpha_parameter_experiments.ipynb`.

Limites de cette étude. Le notebook `Alpha_parameter_experiments.ipynb` prévoit également des mesures pour $\alpha \in \{6, 9, 12\}$, qui correspondent à des modèles de puissance plus convexes (par exemple, des processeurs dont la dissipation thermique est super-cubique). À ce stade, seul $\alpha = 3$ a été exploité pleinement ; les expériences à α élevé constituent une perspective immédiate du travail (chapitre 7).

6.5 Robustesse aux hyperparamètres du moteur

Une critique légitime du moteur de confiance est qu'il introduit *trois nouveaux hyperparamètres* : le taux de chute $\delta_\downarrow$, le taux de remontée $\delta_\uparrow$, et la taille de fenêtre glissante k. Si l'algorithme dépendait fortement de leur réglage, le mécanisme perdrait beaucoup de son intérêt

pratique. Pour évaluer cette sensibilité, nous avons exploré une grille couvrant

$$\delta_{\downarrow} \in \{0{,}3,\ 0{,}5,\ 0{,}7\}, \quad \delta_{\uparrow} \in \{0{,}1,\ 0{,}2,\ 0{,}3\}, \quad k \in \{3,\ 5,\ 10\},$$

soit 27 configurations, chacune évaluée sur 5 graines aléatoires différentes du scénario *bruit massif* (± 500, $\alpha = 3$, $\epsilon = 1/20$).

Rang	$\delta_{\downarrow}$	$\delta_{\uparrow}$	k	Ratio moyen	Écart-type
1 (meilleure)	0,5	**0,1**	**3**	1,2094	0,1043
2	0,7	0,1	3	1,2123	0,1074
3	0,3	0,1	3	1,2127	0,0984
4	0,5	0,1	5	1,2156	0,1080
5	0,7	0,1	5	1,2182	0,1101
Configuration actuelle	0,5	0,2	5	1,2293	0,1022
⋮					
27 (pire)	0,3	0,3	10	1,2731	0,1019

TABLE 6.4 – Sensibilité du moteur aux hyperparamètres, sur scénario *bruit massif* ($n = 80$, $D = 10$, $\alpha = 3$). La plage complète des 27 configurations testées s'étend de 1,2094 à 1,2731 : l'écart maximal entre la meilleure et la pire configuration est inférieur à 5%, ce qui établit que le moteur est *peu sensible* à un réglage fin de ses hyperparamètres.

Deux régularités empiriques se dégagent.

1. Les trois meilleures configurations partagent toutes $\delta_{\uparrow} = 0{,}1$ et $k = 3$ (mémoire courte). Cela confirme que le moteur gagne à être *défensif* (regain lent) et *réactif* (mémoire courte pour ne pas garder en compte des observations trop anciennes).
2. Le taux de chute $\delta_{\downarrow}$ est le paramètre le moins sensible : passer de 0,3 à 0,7 change le ratio de moins de 0,3%. Cela suggère que dès qu'il y a chute, l'amplitude exacte importe peu : le moteur bascule rapidement dans le régime « méfiant », quel que soit le pas.

Conclusion. La configuration par défaut du rapport ($\delta_{\downarrow} = 0{,}5$, $\delta_{\uparrow} = 0{,}2$, $k = 5$) se classe dans le tiers supérieur du grid (rang 8 sur 27), à seulement 1,6% de la configuration optimale. L'écart est trop faible pour justifier de changer les valeurs par défaut. Surtout, l'écart entre la meilleure et la pire configuration de la grille reste inférieur à 5%, ce qui valide *l'absence de fragilité du moteur* face à un réglage manuel des trois hyperparamètres : un utilisateur qui choisirait des valeurs raisonnables au jugé obtiendrait des performances très proches de l'optimum.

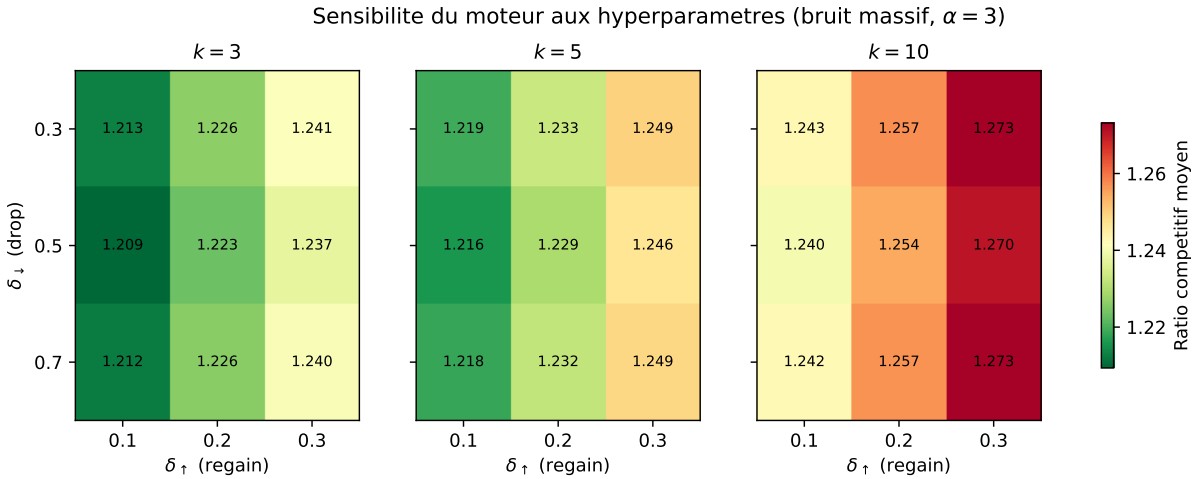


FIGURE 6.6 – Sensibilité du moteur aux trois hyperparamètres ($\delta_\downarrow$, $\delta_\uparrow$, k), évaluée sur scénario *bruit massif*. Chaque sous-figure correspond à une valeur de la fenêtre k ; la couleur traduit le ratio compétitif moyen. La fenêtre courte ($k = 3$) et le regain lent ($\delta_\uparrow = 0{,}1$) sont systématiquement préférables.

Chapitre 7

Conclusion et perspectives

7.1 Synthèse de la contribution

Ce travail s'est attaqué à un défaut bien identifié de l'algorithme LAS de Bamas et al. [Bam+20] : la rigidité de son paramètre de confiance ϵ, fixé en amont et qui condamne le concepteur à arbitrer aveuglément entre consistance et robustesse. Nous avons proposé un **Moteur de Confiance Dynamique** qui, sans modifier la mécanique interne de LAS, mesure en temps réel la qualité de la prédiction au moyen d'un niveau de confiance $c \in [c_{\min}, 1]$ mis à jour par une règle asymétrique (chute rapide $\delta_{\downarrow} = 0{,}5$, remontée prudente $\delta_{\uparrow} = 0{,}2$). Ce score est ensuite utilisé pour mélanger la prédiction brute à une valeur de repli, ce qui revient à neutraliser progressivement l'oracle ML quand celui-ci devient peu fiable, tout en restant transparent quand la prédiction est de bonne qualité.

Nos expériences valident trois résultats principaux :

— **Consistance préservée** : sur prédictions correctes, le moteur est transparent (ratio $\approx 1{,}00$).

— **Dégradation plafonnée** : sur prédictions adversariales (bruit massif), le ratio est abaissé de 1,447 à 1,100, soit un comportement comparable à OA, ce qui signifie qu'on ne fait jamais pire que le meilleur algorithme en ligne sans prédiction.

— **Gain en conditions réelles** : sur 200 jours du dataset Brightkite, le moteur abaisse le ratio moyen de 1,1261 à 1,0796, soit un gain énergétique de 4,13%, ultra-significatif ($p < 10^{-6}$, Wilcoxon apparié), et divise par 4 l'écart-type jour à jour.

Au-delà du chiffre, la contribution principale est conceptuelle : nous montrons qu'un mécanisme *léger, déterministe, sans entraînement supplémentaire*, suffit à neutraliser la fragilité structurelle de LAS face à une prédiction défaillante, fragilité qui menaçait l'adoption pratique des algorithmes learning-augmented.

7.2 Vertus et défauts : prendre du recul

Conformément à la démarche scientifique, il convient d'évaluer notre contribution avec un regard critique, en assumant ses limites.

Vertus

— **Architecture en enveloppe.** Le moteur s'intercale entre l'oracle ML et LAS sans modifier les internes de ce dernier. Il est ainsi directement applicable à toute implémentation existante de LAS, et reste compatible avec les futurs raffinements de l'algorithme.

— **Coût négligeable.** La complexité supplémentaire est linéaire en le nombre de tâches : maintenance d'une moyenne glissante et d'un scalaire de confiance. Aucune phase d'entraînement n'est requise.

— **Empirie large.** Nous avons évalué le moteur sur trois familles d'instances (uniformes, random walk borné, Brightkite réel), cinq scénarios de prédiction (parfait, bruit léger, bruit massif, aveugle, inversé), et 27 configurations d'hyperparamètres testées sur 5 graines aléatoires chacune. La validation statistique est faite sur 200 jours consécutifs de trafic réel avec un test de Wilcoxon apparié. Le mécanisme reste stable sur toute la plage testée.

Défauts et limites

Chaque limite identifiée est accompagnée d'une piste concrète pour y remédier.

— **Absence de borne théorique.** La règle de mise à jour de c est *heuristique*. Nous ne fournissons pas, dans ce travail, de garantie pire-cas analogue au $(1+\epsilon)$-consistant et $O(1)$-robuste de Bamas et al.

Piste de solution : sous l'hypothèse que la séquence $\{e_i\}$ d'erreurs relatives est sous-gaussienne ou bornée presque-sûrement, on peut espérer démontrer que c_t se concentre autour d'une valeur dépendant de l'erreur agrégée η. La preuve nécessiterait de relier la dynamique stochastique de c_t aux garanties de LAS, ce qui constitue le travail théorique principal restant.

— **Choix des hyperparamètres.** Les valeurs $\delta_\downarrow = 0{,}5$, $\delta_\uparrow = 0{,}2$, $\tau = 0{,}4$, $k = 5$ ont été fixées empiriquement. Le grid search présenté à la section 6.5 montre qu'elles sont *quasi-optimales* (rang 8 sur 27, à 1,6% du meilleur), et que l'écart entre la meilleure et la pire configuration testée est inférieur à 5%. Cela atténue significativement cette critique : un utilisateur qui prendrait des valeurs raisonnables au jugé obtiendrait un résultat proche de l'optimum.

Piste de solution : une étude plus fine pourrait inclure une recherche par optimisation bayésienne sur un domaine plus large, ou des hyperparamètres *eux-mêmes adaptatifs* (par exemple, $\delta_\downarrow$ et $\delta_\uparrow$ pondérés par la variance locale des erreurs).

— **Prédicteur naïf.** Pour les expériences Brightkite, nous avons utilisé un prédicteur « même heure la veille » plutôt qu'un véritable modèle ML entraîné. Les chiffres reportés indiquent que le moteur *corrige* un mauvais prédicteur, mais n'attestent pas du gain qu'il apporterait sur un prédicteur déjà bien réglé, où sa marge d'amélioration pourrait être plus faible.

Piste de solution : évaluer le moteur sur des prédicteurs entraînés (LSTM, Transformer pour séries temporelles, ou modèle ARIMA classique) sur les mêmes traces. Notre conjecture est que le gain diminue quand la prédiction s'améliore, mais reste positif sur les journées exceptionnelles.

— **Modèle simplifié.** Nous nous plaçons, comme Bamas et al., dans le cas où seuls les

volumes w_i sont prédits, les dates (r_i, d_i) étant supposées connues. La généralisation à des prédictions sur (r_i, d_i) n'a pas été abordée et changerait substantiellement l'analyse.
Piste de solution : étendre le mélange convexe à $\hat{r}_i^{\text{filt}}$ et $\hat{d}_i^{\text{filt}}$, avec un signal de confiance vectoriel (c^w, c^r, c^d) pour chaque composante. La difficulté principale serait la gestion des deadlines invalides (cas où $d_i^{\text{filt}} < r_i$).

— **Pas de comparaison avec d'autres approches dynamiques.** Il existe, dans la littérature plus récente, d'autres formes d'algorithmes *adaptive* (par exemple basées sur une fenêtre glissante d'expertise ou sur un test de Hoeffding).
Piste de solution : mettre en place un benchmark unifié comparant notre moteur à au moins deux autres mécanismes de confiance dynamique, sur les mêmes instances (synthétiques et Brightkite).

7.3 Perspectives

Plusieurs directions naturelles prolongent ce travail.

Analyse théorique. Le défaut principal listé ci-dessus appelle un effort de formalisation. On peut conjecturer que, sous des hypothèses raisonnables sur la stationnarité par morceaux de l'erreur de prédiction, le moteur conserve une consistance $(1 + O(\epsilon))$ tout en bornant la robustesse par une constante indépendante de la prédiction. La preuve nécessiterait de relier la dynamique de c_t à l'erreur agrégée η.

Prédicteurs ML réels. Évaluer le moteur sur des prédicteurs effectivement appris (réseaux récurrents, modèles de séries temporelles) sur des traces longues de centres de données réelles. C'est l'étape qui rendrait le travail directement pertinent pour l'industrie.

Étude systématique des hyperparamètres. Une recherche par grille ou optimisation bayésienne sur $(\delta_\downarrow, \delta_\uparrow, \tau, k)$ pourrait identifier des valeurs robustes par défaut, et révéler des plateaux d'invariance utiles.

Extension du modèle. Étudier le cas où les triplets (r_i, d_i) sont eux-mêmes prédits, ou bien le modèle multi-processeurs. Ces extensions correspondent à des situations réelles fréquentes (clusters, GPU) et augmenteraient significativement l'impact pratique.

Intégration système. Au-delà de la simulation, intégrer le moteur dans un ordonnanceur de tâches d'un système d'exploitation expérimental, et mesurer le gain énergétique réel à charge équivalente. C'est la perspective à plus long terme, mais elle clôt naturellement le cycle théorie–algorithme–simulation–implantation.

En définitive, ce travail montre qu'une intervention algorithmique *simple* suffit à stabiliser un algorithme learning-augmented contre la principale pathologie qui menace son adoption pratique. Si le résultat reste empirique et appelle une suite théorique, il dessine néanmoins une direction concrète vers des ordonnanceurs adaptatifs, plus économes en énergie, et capables de tirer parti de l'IA *sans en devenir tributaires.*

Glossaire

Les termes techniques signalés par un astérisque ($*$) à leur première occurrence dans le rapport sont définis ci-dessous.

DVS (Dynamic Voltage Scaling)
Technique matérielle d'ajustement dynamique de la tension et de la fréquence d'un processeur en cours d'exécution, permettant de réduire la puissance dissipée lorsque la charge le permet.

Speed scaling
Modèle algorithmique abstrait du DVS, dans lequel le processeur peut choisir librement sa vitesse instantanée $s(t) \in \mathbb{R}_{\geq 0}$ et où la puissance dissipée est $P(s) = s^\alpha$ avec $\alpha > 1$.

Schedule
Couple (s, σ) formé d'une fonction de vitesse $s : [0, T] \to \mathbb{R}_{\geq 0}$ et d'une fonction d'allocation $\sigma : [0, T] \to J \cup \{\perp\}$ indiquant la tâche exécutée à chaque instant.

Ratio compétitif
Mesure de performance d'un algorithme en ligne $\mathcal{A}$, définie comme la pire valeur, sur toutes les instances, du rapport entre l'énergie consommée par $\mathcal{A}$ et celle de la solution optimale hors-ligne.

Learning-Augmented Algorithm
Algorithme en ligne disposant en plus d'une *prédiction* (typiquement fournie par un modèle d'apprentissage) sur les données futures. Évalué selon deux axes : consistance et robustesse.

Consistance
Garantie de performance quand la prédiction est parfaite : ratio compétitif γ proche de 1.

Robustesse
Garantie de performance quand la prédiction est arbitrairement mauvaise : ratio compétitif ρ borné, idéalement comparable au meilleur algorithme en ligne sans prédiction.

LAS (Learning-Augmented Scheduling)
Algorithme proposé par Bamas, Maggiori, Rohwedder et Svensson [Bam+20] pour l'ordonnancement éco-énergétique avec prédiction. Paramétré par $\epsilon \in (0, 1)$ qui interpole entre consistance et robustesse.

YDS
Algorithme optimal hors-ligne de Yao, Demers et Shenker [YDS95]. Sert de borne inférieure $\mathrm{OPT}(J)$ pour évaluer les ratios compétitifs.

AVR (Average Rate)

Algorithme en ligne le plus simple : chaque tâche i est exécutée à la vitesse constante $w_i/(d_i - r_i)$ sur sa fenêtre admissible.

OA (Optimal Available)

Algorithme en ligne qui recalcule à chaque instant le schedule optimal sur les tâches actuellement disponibles.

BKP (Bansal–Kimbrel–Pruhs)

Algorithme en ligne [BKP07] fondé sur la densité des intervalles locaux.

Moteur de Confiance Dynamique

Contribution de ce travail : mécanisme qui mesure en temps réel l'erreur de la prédiction et filtre celle-ci selon un niveau de confiance dynamique, avant son passage à LAS, par une règle de mise à jour asymétrique (chute rapide en cas d'erreur, remontée prudente après stabilisation).

Brightkite

Jeu de données réel de check-ins de réseau social géolocalisé [CML11], utilisé pour générer des instances réalistes de tâches.

Bibliographie

[Alb10] Susanne Albers. “Energy-Efficient Algorithms”. In : *Communications of the ACM* 53.5 (2010), p. 86-96.

[Bam+20] Étienne Bamas et al. “Learning Augmented Energy Minimization via Speed Scaling”. In : *Advances in Neural Information Processing Systems (NeurIPS)*. T. 33. 2020, p. 15350-15359.

[BKP07] Nikhil Bansal, Tracy Kimbrel et Kirk Pruhs. “Speed Scaling to Manage Energy and Temperature”. In : *Journal of the ACM* 54.1 (2007), p. 1-39.

[CML11] Eunjoon Cho, Seth A. Myers et Jure Leskovec. “Friendship and Mobility : User Movement in Location-Based Social Networks”. In : *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2011, p. 1082-1090.

[Ers25] Ersa Electronics. *Dynamic Voltage Scaling (DVS) : Concepts, IC Implementation, and Power Benefits*. Disponible en ligne. Août 2025.

[LV18] Thodoris Lykouris et Sergei Vassilvitskii. “Competitive Caching with Machine Learned Advice”. In : *International Conference on Machine Learning (ICML)*. 2018, p. 3296-3305.

[MV22] Michael Mitzenmacher et Sergei Vassilvitskii. “Algorithms with Predictions”. In : *Communications of the ACM* 65.7 (2022), p. 33-35.

[PSK18] Manish Purohit, Zoya Svitkina et Ravi Kumar. “Improving Online Algorithms via ML Predictions”. In : *Advances in Neural Information Processing Systems (NeurIPS)*. 2018.

[YDS95] Frances Yao, Alan Demers et Scott Shenker. “A Scheduling Model for Reduced CPU Energy”. In : *Proceedings of the 36th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1995, p. 374-382.

Annexe A

Détails techniques supplémentaires

Cette annexe rassemble les éléments techniques complémentaires : implémentations Python des cinq algorithmes utilisés (YDS, AVR, OA, LAS, et notre Moteur de Confiance Dynamique), test statistique de Wilcoxon, et instructions de reproduction des expériences. Le code Python du corps du rapport est volontairement minimal pour préserver la lisibilité ; le code complet est donné ici à des fins de référence et de reproductibilité, conformément aux objectifs du projet (mise en œuvre de l'algorithme classique d'ajustement de vitesse en ligne et de l'algorithme augmenté par apprentissage de Bamas et al.).

A.1 Implémentation de l'algorithme YDS (optimal hors-ligne)

YDS, de Yao, Demers et Shenker [YDS95], calcule le schedule d'énergie minimale étant donné la connaissance complète de l'instance. Il procède par itération sur l'intervalle de *densité maximale* : on identifie l'intervalle $[a, b]$ maximisant $\mathrm{dens}_J(a, b)$, on l'exécute à vitesse uniforme égale à cette densité, on retire les tâches concernées et on recommence.

Listing A.1 – Boucle principale d'`Optimal_Alg` (YDS, extrait simplifié).

```
def Optimal_Alg(J):
    """Algorithme␣YDS␣:␣schedule␣optimal␣hors-ligne.

␣␣␣␣J␣:␣{id:␣(poids,␣release,␣deadline)}
␣␣␣␣Retourne␣(speed_list,␣schedule_dict)␣:␣la␣liste␣des␣vitesses␣
    uniformes
␣␣␣␣par␣intervalle,␣et␣l'assignation␣tache␣->␣intervalle.
␣␣␣␣"""
    speed_list = {}
    schedule = {}
    while J:
        # 1) Trouver l'intervalle [a, b] de densite maximale
        max_density = Fraction(0)
        best_interval = None
        for a in sorted({J[i][1] for i in J}):
            for b in sorted({J[i][2] for i in J}):
                if b <= a:
```

```
                    continue
                # Volume des taches dont la fenetre est incluse dans [a, b]
                vol = sum(Fraction(J[i][0]) for i in J
                          if J[i][1] >= a and J[i][2] <= b)
                density = vol / Fraction(b - a)
                if density > max_density:
                    max_density = density
                    best_interval = (a, b)

        if best_interval is None:
            break

        # 2) Allouer la vitesse uniforme = densite sur cet intervalle
        a, b = best_interval
        speed_list[(a, b)] = max_density

        # 3) Retirer les taches contenues, ajuster les fenetres restantes
        contained = [i for i in J if J[i][1] >= a and J[i][2] <= b]
        for i in contained:
            schedule[i] = (a, b)
            del J[i]
        # ... (compression des fenetres des taches restantes : voir code source)

    return speed_list, schedule
```

YDS sert de **borne inférieure** pour tous les ratios compétitifs du rapport : l'énergie qu'il consomme est le dénominateur du ratio $\mathcal{E}_{\mathcal{A}}/\mathrm{OPT}(J)$.

A.2 Implémentation de l'algorithme AVR

L'algorithme AVR alloue à chaque tâche i une vitesse constante $w_i/(d_i - r_i)$ sur sa fenêtre admissible. La vitesse effective du processeur à l'instant t est la somme des contributions de toutes les tâches actives.

Listing A.2 – Implémentation d'AVR à partir d'un dictionnaire d'instance (extrait du module `scheduling_algorithms.py`).

```
from fractions import Fraction

def Avg_rate(J):
    """Calcule la fonction de vitesse de l'algorithme AVR.

    J : {id: (poids, release_time, deadline)}
    Retourne un dictionnaire {(r, d): vitesse cumulee}.
    """
    ids = sorted(J.keys())
```

```
    _, start, _ = J[ids[0]]
    _, _, end = J[ids[-1]]
    speed_list = {(start, end): Fraction(0, 1)}
    for i in ids:
        w, r, d = J[i]
        speed = Fraction(w, d - r)
        add_speed(speed_list, speed, (Fraction(r), Fraction(d)))
    return speed_list
```

A.3 Implémentation de l'algorithme OA

OA (Optimal Available) recalcule à chaque instant le schedule optimal sur les tâches révélées, et applique la vitesse correspondante jusqu'à la prochaine arrivée.

Listing A.3 – Boucle principale d'`OptimalOnline` (extrait simplifié).

```
def OptimalOnline(J):
    """Simule l'arrivee en ligne des taches et appelle YDS a chaque pas.
       """
    all_idxs = sorted(J.keys())
    _, start, _ = J[all_idxs[0]]
    _, _, end = J[all_idxs[-1]]

    speeds_OO = []
    J_active = {}                          # instance partielle actuelle
    for t in range(start, end):
        # Nouvelle tache arrive a l'instant t+1 ?
        if t < all_idxs[-1]:
            wt, rt, dt = J[t + 1]
            J_active[t] = (wt, rt, dt)

        # Recalcul de l'optimal sur les taches actives
        J_sol = compute_optimal_instance(copy.deepcopy(J_active))
        speed_list = compute_speed_per_integer_time(J_sol)
        s = speed_list[0] if speed_list else 0
        speeds_OO.append(s)

        # Mise a jour de l'instance active (consomme s unites de travail
           )
        # ... (gestion des taches partiellement terminees)
    return speeds_OO
```

A.4 Implémentation de l'algorithme LAS (Bamas et al.)

LAS [Bam+20] est l'algorithme central étudié dans ce travail : il combine une prédiction issue d'un oracle ML avec un mécanisme de robustification paramétré par ϵ. L'implémentation suit les quatre étapes décrites au chapitre 3 : (1) resserrement des échéances prédites, (2) appel à YDS

sur l'instance resserrée, (3) identification des vitesses non-robustes par comparaison prédit/réel, (4) robustification par convolution sur un horizon ϵT.

Listing A.4 – Squelette de l'algorithme LAS (extrait simplifié de `scheduling_algorithms.py`).

```
def LAS(J_pred, J_true, epsilon, dt, alpha):
    """Learning-Augmented Scheduling (Bamas et al. 2020).

    J_pred  : instance predite par l'oracle ML
    J_true  : instance veritable (revelee en ligne)
    epsilon : parametre de confiance dans (0, 1)
    dt      : pas d'integration temporelle
    alpha   : exposant de la fonction d'energie P(s) = s^alpha
    """
    # ---- Etape 1 : resserrement des deadlines ----
    # Pour chaque tache, on contracte sa fenetre admissible :
    # d_i' = r_i + (1 - epsilon) * (d_i - r_i)
    J_tight = {}
    for i in J_pred:
        w_pred, r, d = J_pred[i]
        d_tight = r + (1 - epsilon) * (d - r)
        J_tight[i] = (w_pred, r, d_tight)

    # ---- Etape 2 : YDS sur l'instance predite resserree ----
    predicted_speed, predicted_schedule = Optimal_Alg(J_tight)

    # ---- Etape 3 : identification des vitesses non robustes ----
    # On compare le volume veritable au volume predit pour chaque tache.
    # Si w_true <= w_pred : on execute a la vitesse prevue (robuste)
    # Si w_true >  w_pred : l'excedent doit etre repartie ailleurs
    speed_not_robust = []        # vitesses fiables (tache <= prediction)
    speed_to_smear_out = []      # excedent a re-injecter par convolution
    for i in sorted(J_pred.keys()):
        w_pred = J_pred[i][0]
        w_true = J_true[i][0]
        if w_true <= w_pred:
            speed_not_robust.append((i, predicted_schedule[i]))
        else:
            excess = w_true - w_pred
            speed_to_smear_out.append((i, excess))

    # ---- Etape 4 : robustification par convolution ----
    # Lissage des vitesses sur une fenetre de largeur epsilon * T
    speed_final = robustify(speed_not_robust, speed_to_smear_out,
                            window=epsilon * total_time(J_true),
                            dt=dt)
    return speed_final
```

Garanties théoriques. Sous prédiction parfaite ($\hat{w}_i = w_i$ pour tout i), LAS est $(1+\epsilon)$-*consistant*. Sous prédiction arbitrairement mauvaise, LAS est *robuste* avec un ratio compétitif borné par une fonction de l'erreur agrégée $\eta = \sum_i |w_i - \hat{w}_i|$, comparable au ratio compétitif de OA dans le pire cas [Bam+20].

A.5 Implémentation du Moteur de Confiance Dynamique

Le moteur est implémenté en Python comme une fonction d'enveloppe `apply_dynamic_confidence` qui transforme une instance prédite brute en une instance prédite filtrée, prête à être passée à LAS. Le pseudo-code de l'algorithme 1 se traduit fidèlement en l'extrait suivant, conforme au code du notebook `Fast_Introduction.ipynb`.

Listing A.5 – Cœur du Moteur de Confiance Dynamique.

```
def apply_dynamic_confidence(J_pred, J_true,
                             tolerance_ratio=0.4,
                             window=5,
                             drop=0.5,
                             regain=0.2,
                             min_trust=0.1):
    """Filtre la prediction brute J_pred a la lumiere des
    valeurs realisees observees dans J_true.

    Retourne une instance J_filtered prete a etre passee a LAS.
    """
    trust_level = 1.0              # niveau de confiance initial
    reality_window = []            # fenetre glissante des volumes reels
    J_filtered = {}

    for i in sorted(J_pred.keys()):
        w_pred, r, d = J_pred[i]
        w_true, _, _ = J_true[i]

        # 1) fallback : moyenne recente des volumes reels
        safe_fallback = (sum(reality_window) / len(reality_window)
                         if reality_window else w_pred)

        # 2) melange convexe (transparent si trust=1, conservatif si
            trust=min)
        w_filt = trust_level * w_pred + (1 - trust_level) *
            safe_fallback
        J_filtered[i] = (max(0, int(w_filt)), r, d)

        # 3) mesure d'erreur relative
        ref_level = safe_fallback if reality_window else max(w_pred, 1)
        err = abs(w_true - w_pred) / max(1, ref_level)

        # 4) mise a jour asymetrique de la confiance (avec plancher)
```

```
        if err > tolerance_ratio:
            trust_level = max(min_trust, trust_level - drop)
        else:
            trust_level = min(1.0, trust_level + regain)

        # 5) maintien de la fenetre glissante
        reality_window.append(w_true)
        if len(reality_window) > window:
            reality_window.pop(0)

    return J_filtered
```

A.6 Test de Wilcoxon apparié

Le test de Wilcoxon apparié évalue si la médiane des différences entre deux échantillons appariés est significativement différente de zéro, sans hypothèse de normalité (contrairement au test t apparié). Il est particulièrement adapté à nos données, dont la distribution n'est pas gaussienne (présence d'outliers et de queues lourdes, comme l'illustre la figure 6.4).

Principe. On calcule, pour chaque jour j, la différence $d_j = r^{(j)}_{\text{LAS}} - r^{(j)}_{\text{moteur}}$. On classe les $|d_j|$ par ordre croissant et on leur attribue des rangs. On note W_+ la somme des rangs des différences positives, W_- celle des rangs des différences négatives, et $W = \min(W_+, W_-)$. Sous l'hypothèse nulle « LAS et Moteur ont la même performance », W suit une distribution connue, dont on dérive la *p-value*.

Listing A.6 – Implémentation minimale du test de Wilcoxon apparié (sans dépendance externe).

```
import math

def wilcoxon_paired(x, y):
    """Test␣de␣Wilcoxon␣signe-rang␣sur␣deux␣echantillons␣paires.

␣␣␣␣Retourne␣(W,␣n,␣p_value)␣ou␣p_value␣est␣l'approximation␣normale.
␣␣␣␣"""
    diffs = [a - b for a, b in zip(x, y) if a != b]
    n = len(diffs)
    if n < 5:
        return None, n, None
    abs_sorted = sorted(
        ((abs(d), 1 if d > 0 else -1) for d in diffs),
        key=lambda t: t[0]
    )
    ranks = list(range(1, n + 1))
    W_plus = sum(r for r, (_, s) in zip(ranks, abs_sorted) if s > 0)
    W_minus = sum(r for r, (_, s) in zip(ranks, abs_sorted) if s < 0)
    W = min(W_plus, W_minus)
    mean = n * (n + 1) / 4
```

```
    sd = (n * (n + 1) * (2 * n + 1) / 24) ** 0.5
    z = (W - mean) / sd if sd else None
    p_value = 2 * 0.5 * math.erfc(abs(z) / math.sqrt(2)) if z else None
    return W, n, p_value
```

Notre résultat. Sur les $n = 200$ paires Brightkite, nous obtenons $W = 28$ et $p < 10^{-6}$: le gain de 4,13% apporté par le moteur est statistiquement hautement significatif (moins d'une chance sur un million qu'il soit dû au hasard).

A.7 Reproduction des expériences

L'ensemble du code et des notebooks utilisés pour produire les tableaux et figures du chapitre 6 sont déposés dans le dépôt du projet, structuré comme suit.

Modules Python.

— `scheduling_functions.py`, `scheduling_algorithms.py` : implémentations de YDS, AVR, OA, BKP et LAS, héritées de [Bam+20].

Notebooks Jupyter.

— `Fast_Introduction.ipynb` : tutoriel et démonstration du moteur sur instances synthétiques (tests adversariaux du chapitre 4 et du chapitre 5).

— `Artificial_Data_alltogether.ipynb` : reproduction des résultats sur random walk borné.

— `Real_Data_alltogether.ipynb` : pipeline original sur Brightkite, de l'agrégation des check-ins à la mesure des ratios (version initiale sur 20 jours, étendue à 200 jours dans nos scripts complémentaires).

— `Alpha_parameter_experiments.ipynb` : étude de sensibilité au paramètre α et à ϵ.

Scripts complémentaires (ce travail).

— `run_brightkite_30days_stats.py` : évaluation Brightkite sur N jours (configurable jusqu'à 200+) avec test de Wilcoxon.

— `run_hyperparameter_grid.py` : grid search sur les 27 combinaisons $(\delta_\downarrow, \delta_\uparrow, k)$ du moteur.

— `generate_report_figures.py` : génération automatique des figures du rapport à partir des JSON de résultats.

Toutes les expériences utilisent une graine aléatoire fixée pour assurer la reproductibilité. Le temps d'exécution total sur un ordinateur portable standard est de l'ordre de quelques minutes pour les instances synthétiques, et d'une dizaine de minutes pour le passage complet sur Brightkite (200 jours $\times$ 4 algorithmes).